



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

DECLARATION

I understand that this is an individual assessment and that collaboration is not permitted. I have not received any assistance with my work for this assessment. Where I have used the published work of others, I have indicated this with appropriate citation.

I have not and will not share any part of my work on this assessment, directly or indirectly, with any other student.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

I have also completed the Online Tutorial on avoiding plagiarism 'Ready Steady Write', located at <http://tcd-ie.libguides.com/plagiarism/ready-steady-write>."

I understand that by returning this declaration with my work, I am agreeing with the above statement.

Name:

Date:

CS7CS2 – Optimisation for Machine Learning

Student Name: Stephen Komolafe

Student Number: 21336975

Final Assignment

Benchmarks

A set of benchmark problems are used throughout this assignment. These benchmarks are chosen to represent different types of optimisation landscapes. All optimisation methods in later questions are applied to one or more of the following benchmarks.

Benchmark A: Linear Regression Quadratic Loss

A linear regression problem with a quadratic loss function. A synthetic dataset of $m = 1000$ is generated, where each data point satisfies the equation:

$$y^{(i)} = \theta^{*T} x^{(i)} + \varepsilon^{(i)},$$

with true parameter $\theta^* = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$, and zero-mean Gaussian noise $\varepsilon^{(i)} \sim N(0, 1)$.

The objective function is:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2,$$

which can be written in matrix form as:

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|^2,$$

and the functions gradient being:

$$\nabla J(\theta) = \frac{1}{m} X^T (X\theta - y).$$

This benchmark is convex and quadratic, thus providing a well-conditioned test case where most optimisation methods are expected to converge smoothly.

Benchmark B: Toy Neural Network Quadratic Loss

A two-dimensional nonlinear function defined by

$$f(x_1, x_2) = (x_1 - 1)^2 + 5(x_2 - 2)^2 + \sin(x_1).$$

The gradient is

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2(x_1 - 1) + \cos(x_1) \\ 10(x_2 - 2) \end{bmatrix}.$$

This objective is smooth but non-quadratic due to the sinusoidal term. It introduces mild nonlinearity, making it useful for testing how optimisation methods behave when the objective is not perfectly convex or symmetric.

Benchmark C: Rosenbrock Function

The Rosenbrock function which is defined as:

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2.$$

The gradient is

$$\nabla f(x_1, x_2) = \begin{bmatrix} -2(1 - x_1) - 400x_1(x_2 - x_1^2) \\ 200(x_2 - x_1^2) \end{bmatrix}$$

This function is non-convex and has a narrow-curved valley leading to the global minimum. It is used as a test problem for optimisation algorithms as methods must both locate the valley and then move efficiently along it.

Benchmark D: Constrained Feasible Set

In later questions, constrained optimisation problems are considered using a two-dimensional feasible set X , defined as a convex polygon with vertices:

$$X = \{x \in \mathbb{R}^2: 0.5 \leq x_1 \leq 5, \quad -5 \leq x_2 \leq 10\},$$

This defines a box-constrained region in $\mathbb{R}^2$, whose vertices are:

$$(0.5, -5), (5, -5), (5, 10), (0.5, 10).$$

This feasible region can also be expressed using simple bound constraints on each coordinate. This benchmark is used to evaluate constrained optimisation methods, where the geometry of the feasible set plays an important role in determining algorithm behaviour.

Q1:

For this question, the aim was to compare Polyak step size, Adagrad, RMSprop, and Polyak momentum (Heavy Ball) against a constant-step gradient descent baseline on Benchmarks A, B, and C. The benchmark definitions and gradients are given in the Benchmark section above. Each method was run for 120 iterations using the parameter settings specified in the assignment, and both the iterates and objective values were stored for comparison.

(a) Implementing optimisation methods

A constant-step gradient descent baseline was implemented using the update:

$$x^{(t+1)} = x^{(t)} - \alpha \nabla F(x^{(t)}).$$

This was defined in as the function:

```
def gradient_descent_constant(f, grad_f, x0, alpha, n_iters=120):
    x = x0.copy()
    xs, vals = [x.copy()], [f(x)]

    for _ in range(n_iters):
        x = x - alpha * grad_f(x)
        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(vals)
```

Each method was implemented as a separate function so that the same benchmark functions could be reused consistently.

The Polyak step-size method was implemented using

$$\alpha_t = \frac{f(x^{(t)}) - f^*}{\|\nabla f(x^{(t)})\|^2 + \varepsilon},$$

```
def gradient_descent_polyak(f, grad_f, x0, f_star=0.0, eps=1e-4, n_iters=120):
    x = x0.copy()
    xs = [x.copy()]
    vals = [f(x)]
    step_hist = []

    for _ in range(n_iters):
        g = grad_f(x)
        g2 = np.dot(g, g)
        alpha = (f(x) - f_star) / (g2 + eps)
        step_hist.append(alpha)

        x = x - alpha * g
        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(vals), np.array(step_hist)
```

Adagrad used a coordinate-wise accumulator:

```
def adagrad(f, grad_f, x0, alpha0, eps=1e-5, n_iters=120):
    x = x0.copy()
    s = np.zeros_like(x)

    xs = [x.copy()]
    vals = [f(x)]
    step_hist = []

    for _ in range(n_iters):
        g = grad_f(x)
        s = s + g**2
        eff_step = alpha0 / (np.sqrt(s) + eps)
        step_hist.append(eff_step.copy())
        x = x - eff_step * g
        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(vals), np.array(step_hist)
```

RMSprop used exponential averaging of squared gradients:

```
def rmsprop(f, grad_f, x0, alpha0, beta=0.9, eps=1e-5, n_iters=120):
    x = x0.copy()
    s = np.zeros_like(x)

    xs = [x.copy()]
    vals = [f(x)]
    step_hist = []

    for _ in range(n_iters):
        g = grad_f(x)
        s = beta * s + (1.0 - beta) * (g**2)
        eff_step = alpha0 / (np.sqrt(s) + eps)
        step_hist.append(eff_step.copy())

        x = x - eff_step * g
        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(vals), np.array(step_hist)
```

Heavy Ball (momentum) used the update

$$x^{(t+1)} = x^{(t)} - z^{(t+1)}, \text{ where } z^{(t+1)} = \beta z^{(t)} + \alpha \nabla f(x^{(t)}),$$

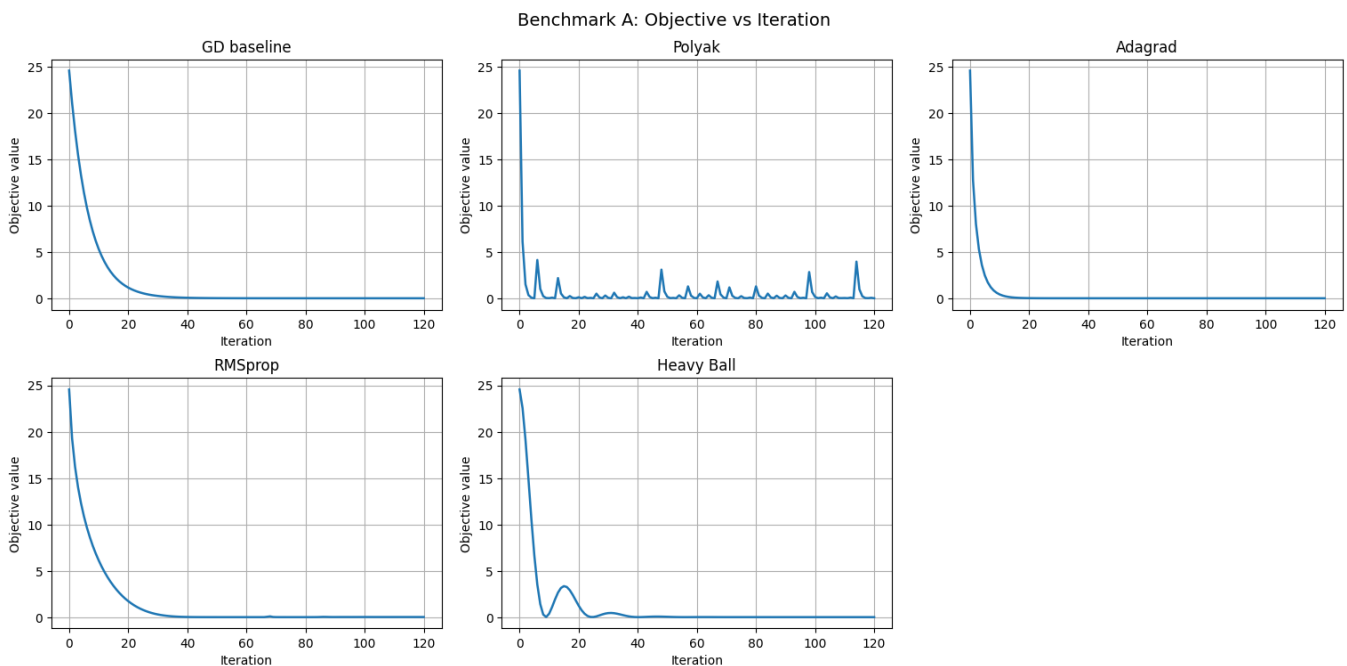
```
def heavy_ball(f, grad_f, x0, alpha, beta, n_iters=120):
    x = x0.copy()
    z = np.zeros_like(x)

    xs = [x.copy()]
    vals = [f(x)]

    for _ in range(n_iters):
        g = grad_f(x)
        z = beta * z + alpha * g
        x = x - z
        xs.append(x.copy())
        vals.append(f(x))

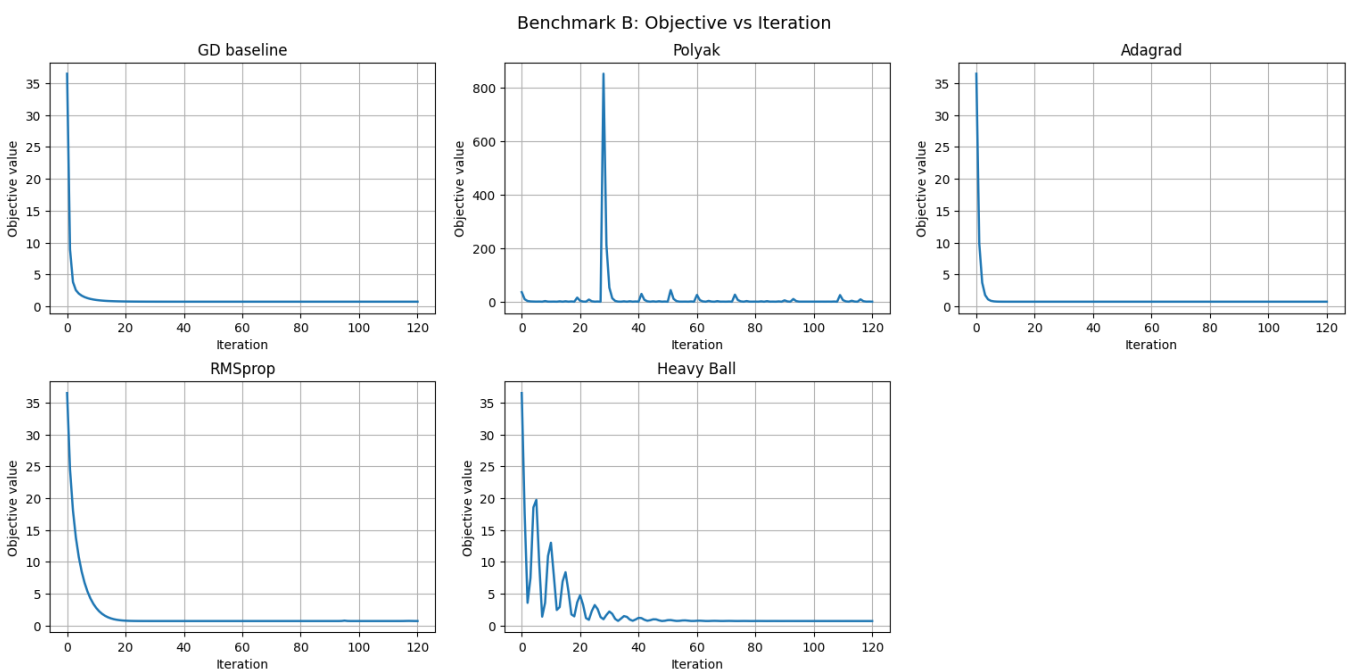
    return np.array(xs), np.array(vals)
```

(b) Objective vs Iteration (for Benchmark A, B, C)



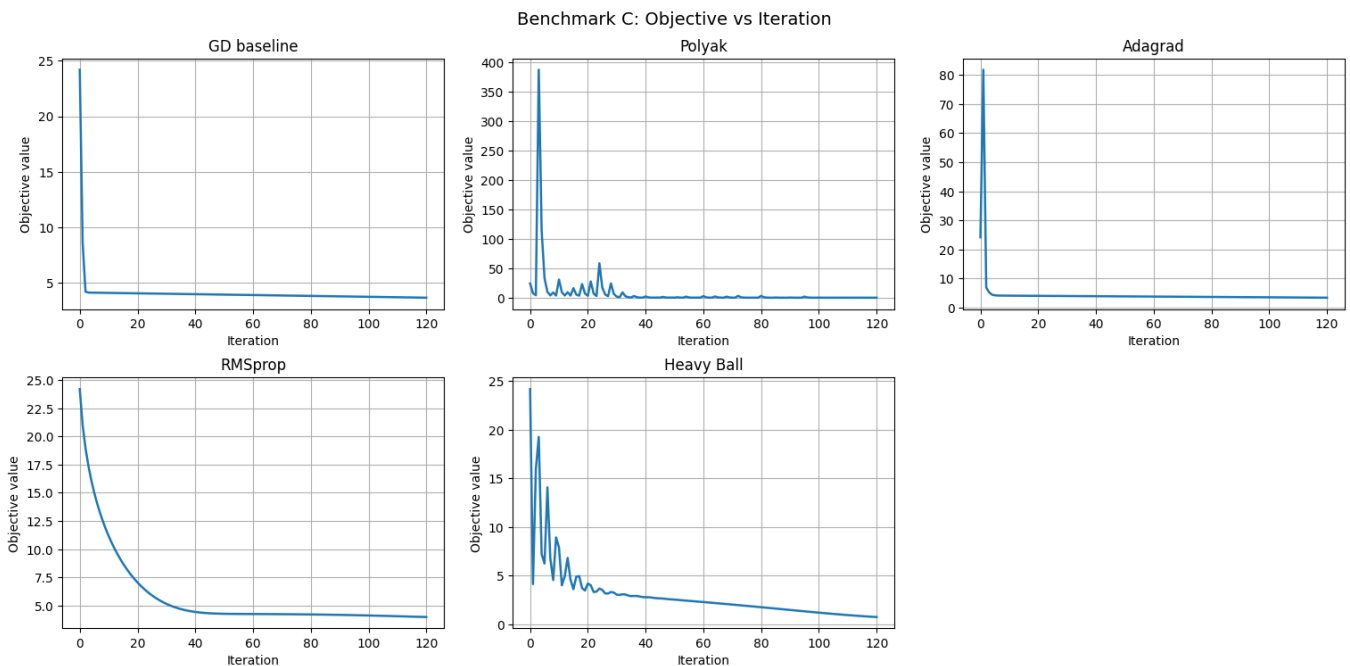
For Benchmark A, all methods converged successfully due to the convex quadratic structure. Gradient descent reduced the objective from 24.555 to 0.062, converging to approximately (3.011, 3.992) on its final iterate. Adagrad and Heavy Ball achieved the same final objective of 0.062, while Polyak reached 0.063 and RMSprop 0.074.

Polyak gave the fastest early decrease, dropping the objective to 6.157, 1.556, and 0.405 within the initial iterations. However, its latter values were less smooth fluctuating between 4.006 and 0.063. In contrast, gradient descent, Adagrad, RMSprop, and Heavy Ball exhibited smooth and stable convergence.



For Benchmark B, gradient descent reduced the objective from 36.503 to 0.72442, converging to (0.582, 2.0). Adagrad matched this value at 0.72442, while Heavy Ball was nearly identical reaching 0.724424, and RMSprop slightly higher at 0.726. Polyak performed worst, finishing at 0.768.

Although Polyak decreased rapidly at first, it was highly unstable, with a maximum objective value of 852.793 during the run. Heavy Ball also showed early oscillation in its objective value but eventually stabilised near the optimum.

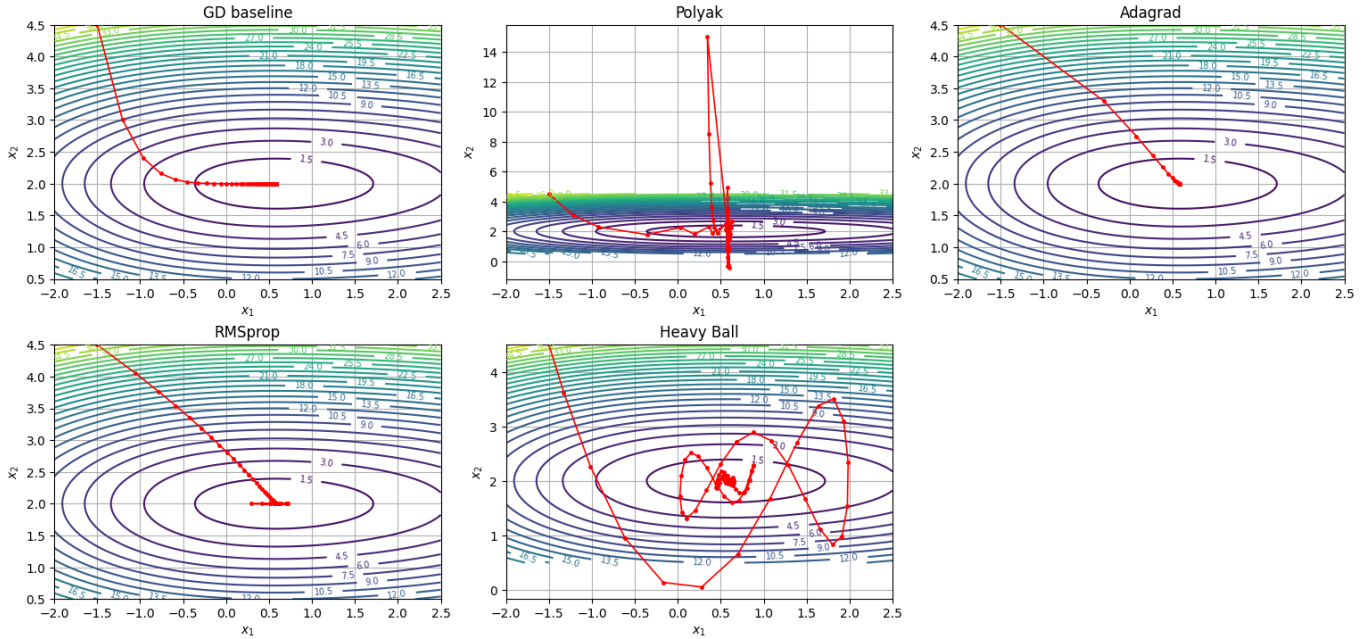


For Benchmark C, the Rosenbrock function was more difficult. Gradient descent reduced the objective from 24.2 to 3.644, while Adagrad reached 3.420 and RMSprop 3.982. Heavy Ball performed notably better, reaching 0.745, and Polyak performed the best overall with a final value of 0.014.

However, Polyak showed extreme instability, with an early spike to 387.98, while Adagrad also spiked to 81.83. RMSprop remained smooth but made slow progress in converging.

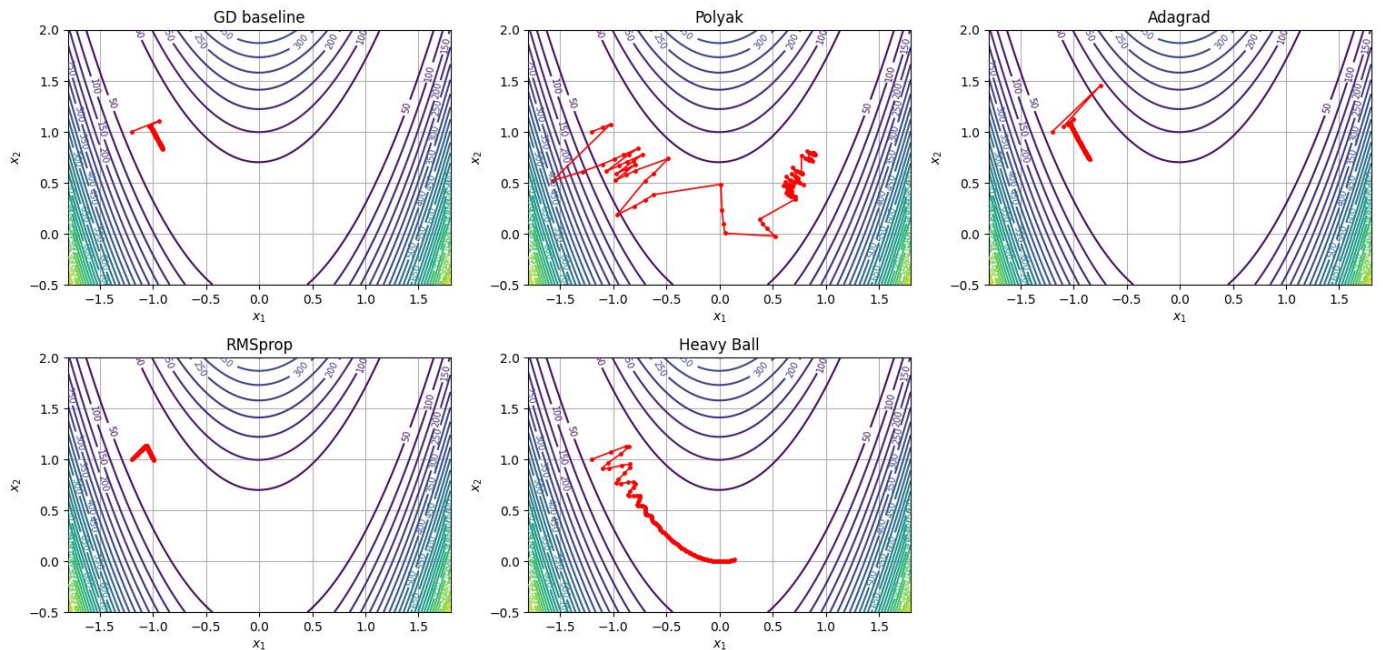
(c) Contour Trajectories

Benchmark B: Contour & Optimisation Trajectories



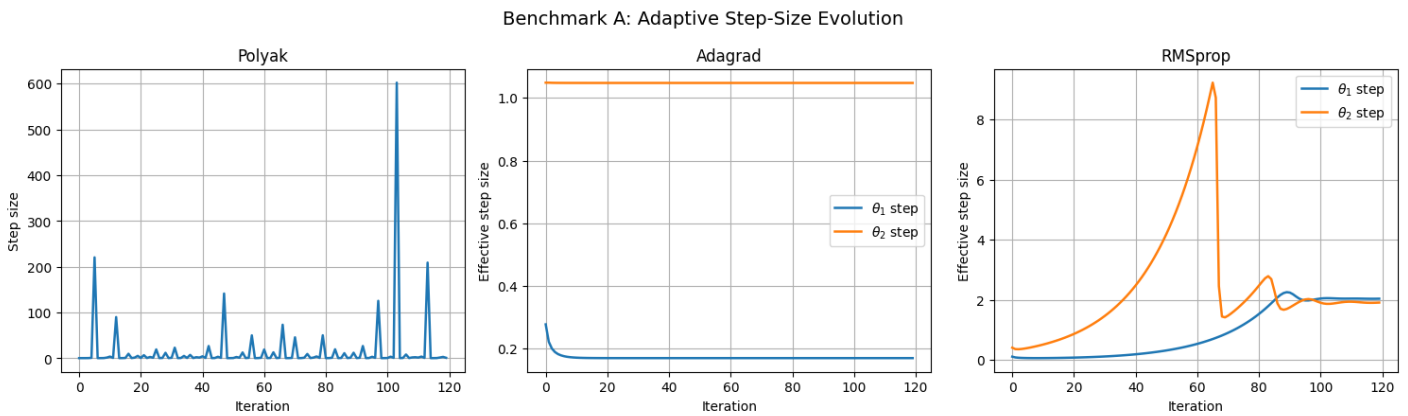
For Benchmark B, the contour plots show that gradient descent and Adagrad follow smooth, direct paths toward the minimiser near (0.582, 2.0). Heavy Ball also converges to this region but with visible oscillation due to momentum. RMSprop approaches the solution but continues to oscillate slightly in the x_1 direction. Polyak shows the most irregular trajectory, particularly in the x_2 direction, where the iterates move above and below the optimal level before settling.

Benchmark C: Contour & Optimisation Trajectories

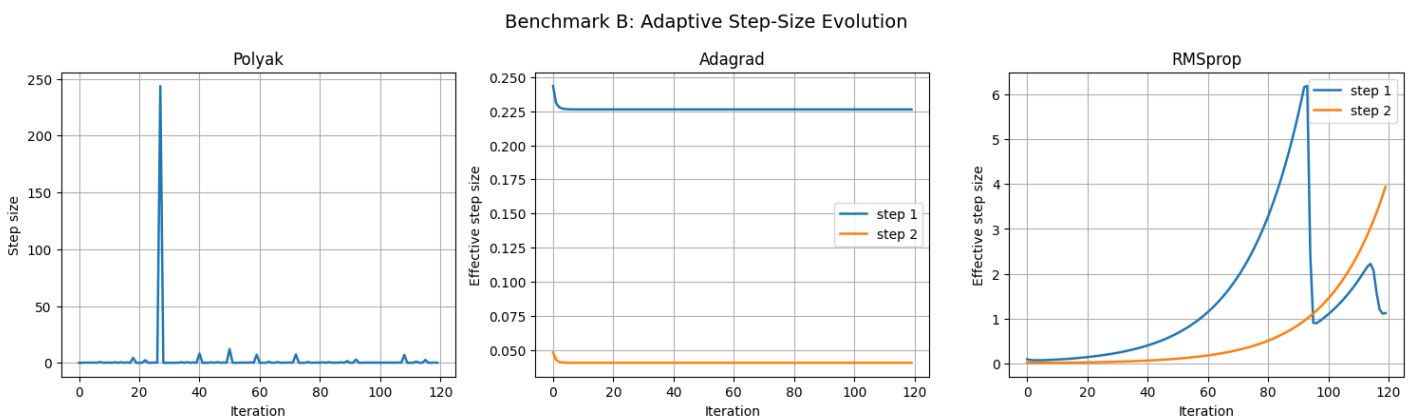


For Benchmark C, the contour plots demonstrate the difficulty of the Rosenbrock valley. Gradient descent and Adagrad move slowly along the curved valley and remain far from the minimiser after 120 iterations. RMSprop behaves similarly but at a slower rate. Heavy Ball moves further along the valley, showing that momentum helps accelerate progress. Polyak produces the most aggressive trajectory, initially overshooting but eventually reaching a region near to the minimum.

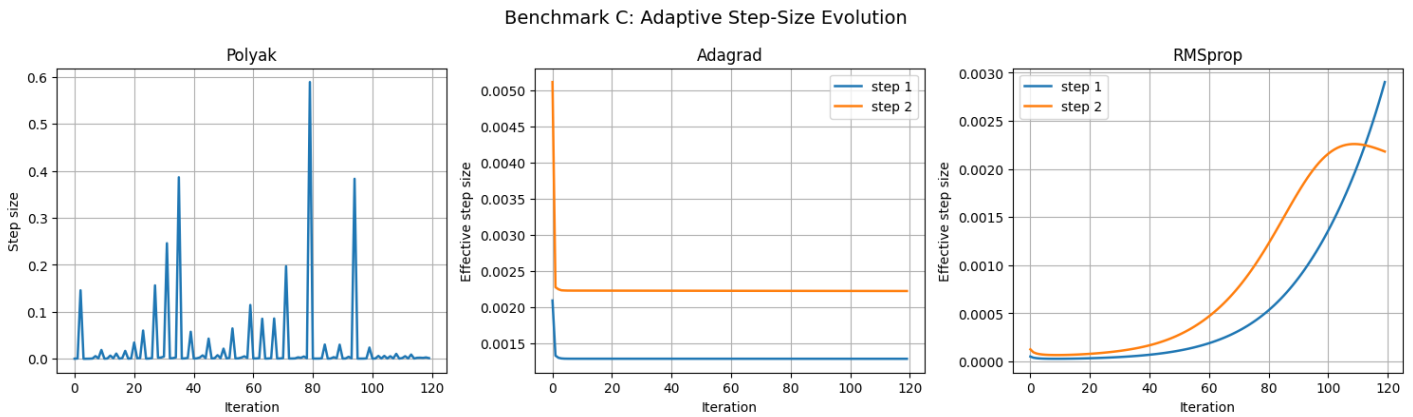
(d) Adaptive step-size evolution



Where (θ_1, θ_2) are the effective step sizes for the first and second coordinates respectively, Benchmark A's Polyak was generally unstable as step size values varied through iterations, ranging from 0.54 to over 600. Adagrad had a smooth decay in effective step sizes going from 0.277 to 0.169 for θ_1 , whereas RMSprop's step size for θ_2 increased gradually, peaking at 9.22, then dropping sharply, fluctuating, and reaching 1.91 upon the final iteration.



Where (x_1, x_2) are the effective step sizes for the first and second coordinates respectively, for Benchmark B, Polyak again showed large variability, with step sizes up to 243.65. Adagrad remained stable, decreasing slightly from (0.243, 0.048) to (0.226, 0.040). RMSprop's step sizes increased sharply, especially in x_2 , reaching 3.94, which explains the oscillations in the trajectory.



Where (x_1, x_2) are the effective step sizes for the first and second coordinates respectively, for Benchmark C, Polyak ranged from 0.0004 to 0.589, causing large jumps in the objective. Adagrad decreased steadily to around (0.0013, 0.0022), giving stable but slow progress. RMSprop started with small step sizes and increased gradually, leading to smooth but slow convergence.

(e) Comparison of convergence speed, smoothness, and stability

Across all benchmarks, gradient descent was the most predictable and stable method, but slow on difficult problems. Polyak provided the fastest early progress and achieved the best result on Rosenbrock (0.014), but was highly unstable, with large objective spikes.

Adagrad was consistently stable and performed well on Benchmarks A and B, matching the best final objective values of 0.062 and 0.724 respectively, but was overly skewed on Rosenbrock at 3.420.

RMSprop was less consistent. It performed reasonably well on simpler problems but showed oscillations and was the weakest on Rosenbrock reaching 3.982 in its final iteration.

Heavy Ball provided a good balance between speed and stability. It matched the best results on Benchmark A, performed well on Benchmark B, and significantly outperformed gradient descent and Adagrad on Rosenbrock (0.745), although with some oscillation.

No single method dominates across all cases. Simpler methods perform best on well-conditioned problems, while more aggressive methods such as Polyak and Heavy Ball are needed to make effective progress on difficult objectives like the Rosenbrock function.

Q2:

For this question, the aim was to compare Nesterov momentum, Adam, and a constant-step gradient descent baseline on Benchmarks A, B, and C, and then to study the behaviour of mini-batch stochastic gradient descent on Benchmark A. In addition, the effect of increased output noise on SGD was analysed by repeating the experiment with a higher noise level in the regression targets. All implementations were carried out in Python using NumPy for computation and Matplotlib for visualisation. Nesterov, Adam, and the baseline were run for 150 iterations, while the SGD experiments were run for 50 epochs, with data shuffling performed at every epoch as required.

(a) Comparing Nesterov, Adam, and the constant baseline on all three benchmarks

The constant-step gradient descent baseline was implemented using the same update as in Q1:

$$x^{(t+1)} = x^{(t)} - \alpha \nabla F(x^{(t)})$$

Nesterov momentum was implemented using the update

$$z^{(t+1)} = \beta_t z^{(t)} - \alpha \nabla f(x^{(t)} + \beta_t z^{(t)}), \quad x^{(t+1)} = x^{(t)} + z^{(t+1)},$$

With

$$\beta_t = \min\left(\frac{t-1}{t+2}, \beta_{\max}\right).$$

This was implemented as the following function:

```
def nesterov_momentum(f, grad_f, x0, alpha, beta_max, n_iters=150):
    x = x0.copy()
    z = np.zeros_like(x)

    xs = [x.copy()]
    vals = [f(x)]

    for k in range(1, n_iters + 1):
        beta_k = min((k - 1) / (k + 2), beta_max)
        lookahead = x + beta_k * z
        g = grad_f(lookahead)

        z = beta_k * z - alpha * g
        x = x + z

        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(vals)
```

Adam was implemented using first and second moment estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

followed by bias correction and the update:

$$x_t = x_{t-1} - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \right),$$

and was implemented as the following function:

```
def adam(f, grad_f, x0, alpha, beta1, beta2, eps=1e-8, n_iters=150):
    x = x0.copy()
    m = np.zeros_like(x)
    v = np.zeros_like(x)

    xs = [x.copy()]
    vals = [f(x)]

    for t in range(1, n_iters + 1):
        g = grad_f(x)

        m = beta1 * m + (1.0 - beta1) * g
        v = beta2 * v + (1.0 - beta2) * (g**2)

        m_hat = m / (1.0 - beta1**t)
        v_hat = v / (1.0 - beta2**t)

        x = x - alpha * m_hat / (np.sqrt(v_hat) + eps)

        xs.append(x.copy())
        vals.append(f(x))

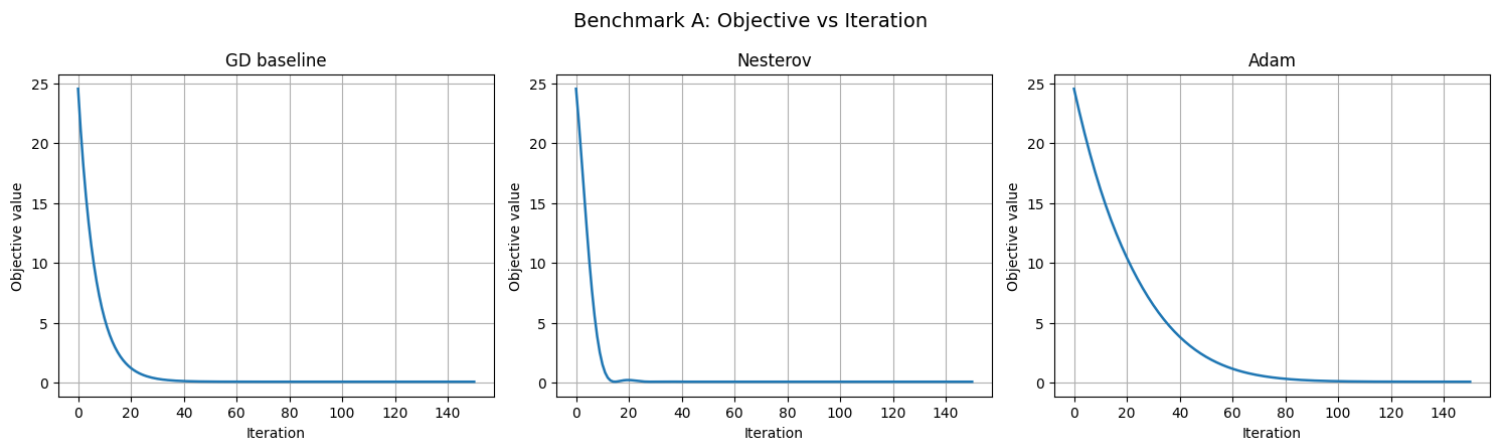
    return np.array(xs), np.array(vals)
```

For Benchmark A, all three methods converged exceptionally. Gradient descent reduced the objective from 24.555 to 0.062, with final iterate approximately (3.011, 3.992). Nesterov produced essentially the same result, also reaching 0.062, while Adam ended slightly higher at 0.062 with final iterate (2.994, 3.993). Thus, on the linear regression benchmark, all three methods performed almost identically by the end of the run.

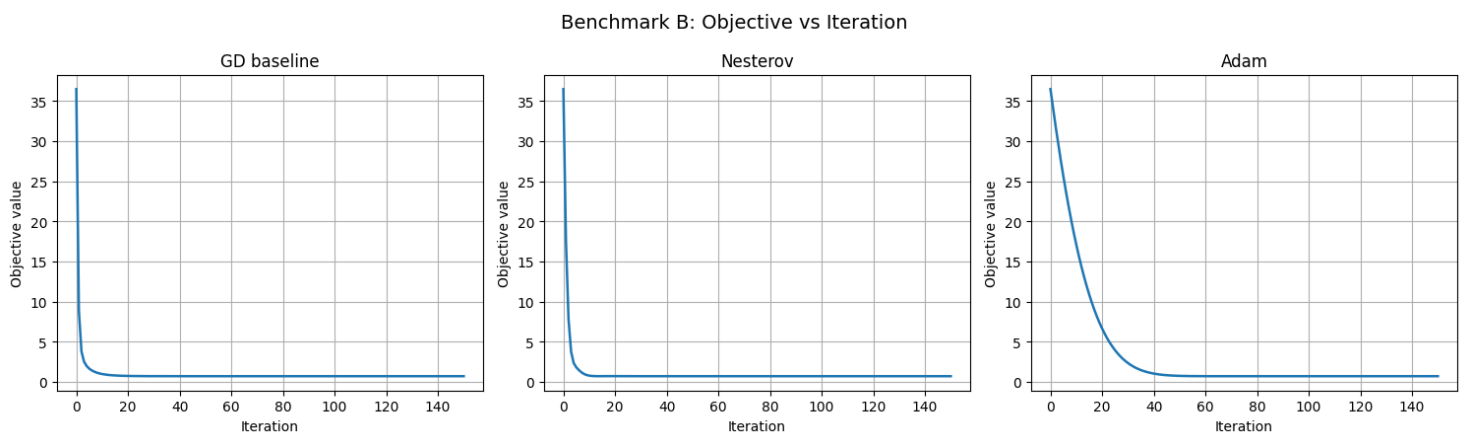
For Benchmark B, all three methods again converged to essentially the same solution. Gradient descent ended at (0.582, 2.0) with final objective 0.7244. Nesterov and Adam both achieved the same final objective value to six decimal places, also reaching the same minimiser numerically. This shows that on the toy nonlinear problem; all three methods were equally effective within 150 iterations.

For Benchmark C, the differences were more significant. Gradient descent reduced the objective from 24.20 to 3.521, while Adam ended at 4.197, which was worse than the baseline. Nesterov, however, performed better, reaching final objective 0.556 and final iterate approximately (0.255, 0.061). Therefore, on the Rosenbrock function, Nesterov clearly outperformed both the baseline and Adam.

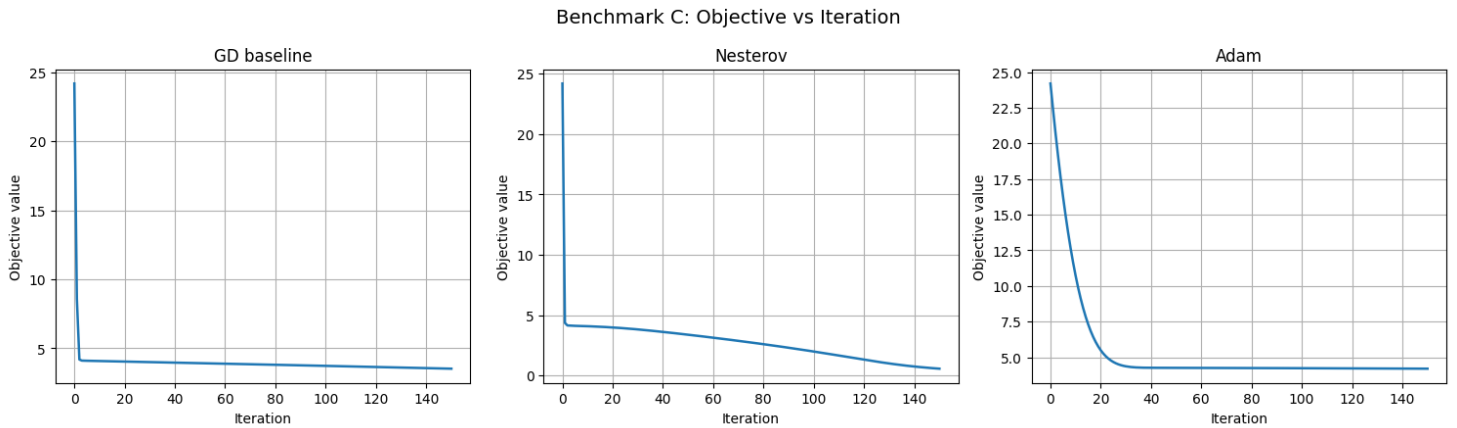
(b) Objective Value vs Iteration



For Benchmark A, gradient descent and Nesterov both decreased the objective quickly and smoothly, eventually flattening at 0.0622. Nesterov was slightly less aggressive than the baseline in the first few steps, with early objective values 24.555, 21.922, 19.005, 15.980, compared with the baseline values 24.555, 21.077, 18.094, 15.534. Adam decreased the objective more gradually at the start, with values 24.555, 23.583, 22.639, 21.724, but it still reached a similar final value. Thus, on Benchmark A, the objective curves suggest that Adam was slower initially, while gradient descent and Nesterov were more direct.

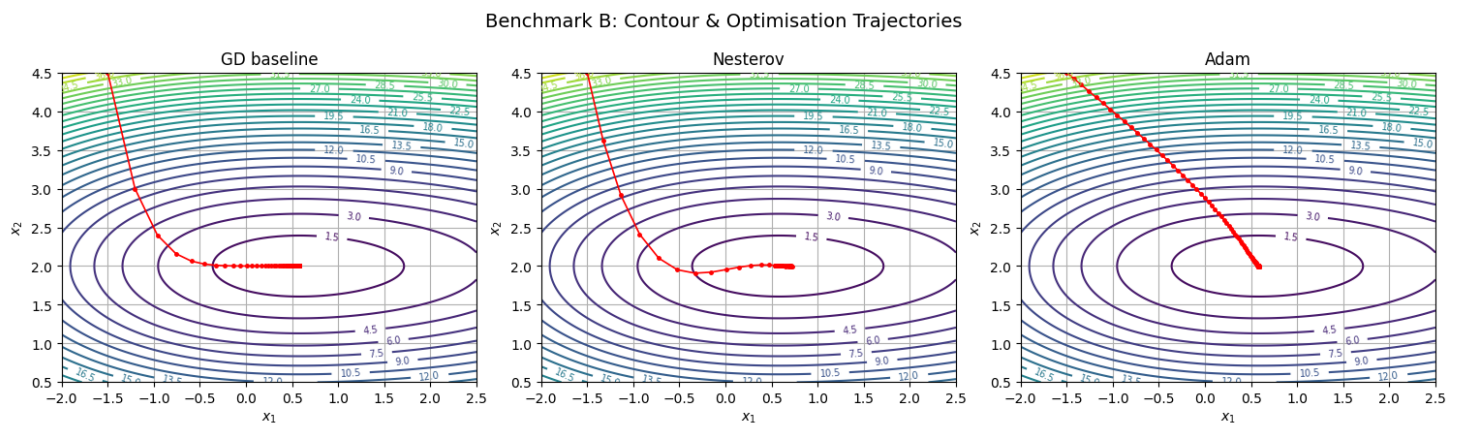


For Benchmark B, gradient descent again gave a fast and smooth decrease, reducing the objective from 36.503 to 8.925, 3.827, 2.537, and 1.998 in the first iterations. Nesterov also converged well, but its early values were somewhat larger: 36.503, 17.650, 7.827, 3.762, 2.362. Adam was clearly the slowest initially, with early values 36.503, 34.150, 31.885, 29.710, 27.628. However, since all three methods eventually reached 0.724, the main difference here is convergence speed rather than final accuracy.



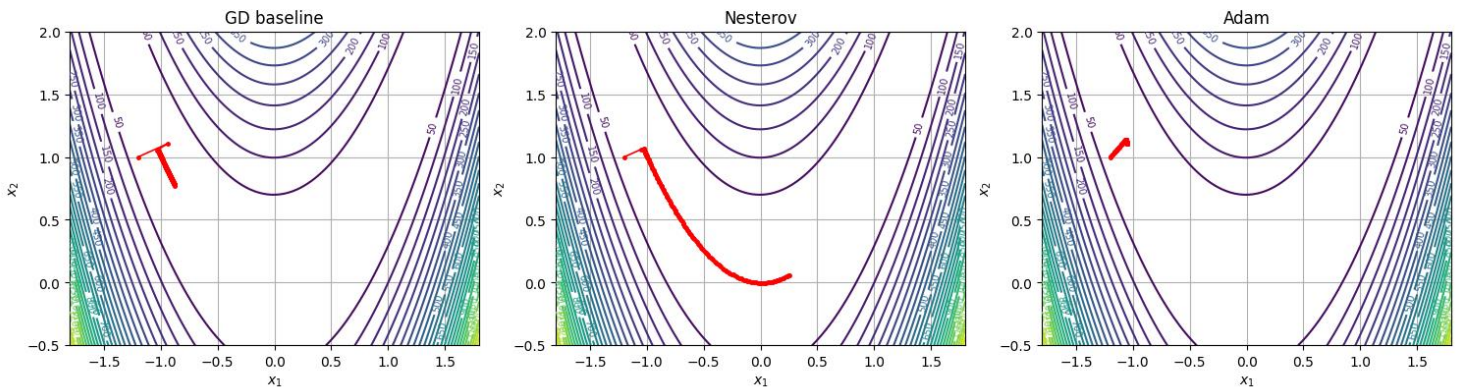
For Benchmark C, the objective plots show the clearest separation. Gradient descent decreased strongly at first, from 24.2 to 8.591 and 4.185, but then slowed down considerably, finishing at 3.521. Nesterov achieved a sharp early drop to 4.351, then continued improving steadily until it reached 0.556. Adam behaved worst on this benchmark, decreasing only gradually from 24.2 to 22.423, 20.74, 19.153, and 17.662 in the first few iterations, before ending at 4.197. Therefore, the objective plots show that Nesterov was the strongest method on Rosenbrock, while Adam was the weakest.

(c) Contour Trajectories for Benchmark B & C



For Benchmark B, the contour plots show that all three methods eventually move toward the same minimiser near (0.582, 2.0). The gradient descent trajectory is smooth and direct. Nesterov also converges to the same region, though its path reflects the momentum effect and is slightly less direct in the early part of the run. Adam's trajectory is slower initially, which is consistent with its more gradual decrease in the objective value, but it still approaches the same minimiser by the end.

Benchmark C: Contour & Optimisation Trajectories



For Benchmark C, the contour plots show a larger difference between the methods. Gradient descent moves slowly along the Rosenbrock valley and remains far from the global minimum after 150 iterations. Adam progresses even more slowly, staying near the left side of the valley and failing to exploit the geometry effectively. Nesterov, by contrast, moves further through the curved valley and reaches a substantially lower objective region. This makes the contour plots fully consistent with the numerical results: Nesterov handles the difficult geometry better than the other two methods.

(d) For SGD, comparing the effect of batch size

Mini-batch SGD was implemented for Benchmark A only, using the update on a shuffled mini-batch at each step and recording the full objective value once per epoch. This was implemented as the following function:

```
def minibatch_sgd_A(X, y, x0, alpha=0.06, batch_size=5, epochs=50, seed=123):
    rng = np.random.default_rng(seed)

    x = x0.copy()
    xs = [x.copy()]
    vals = [f_A(x, X, y)]

    m = X.shape[0]

    for _ in range(epochs):
        perm = rng.permutation(m)
        X_shuf = X[perm]
        y_shuf = y[perm]

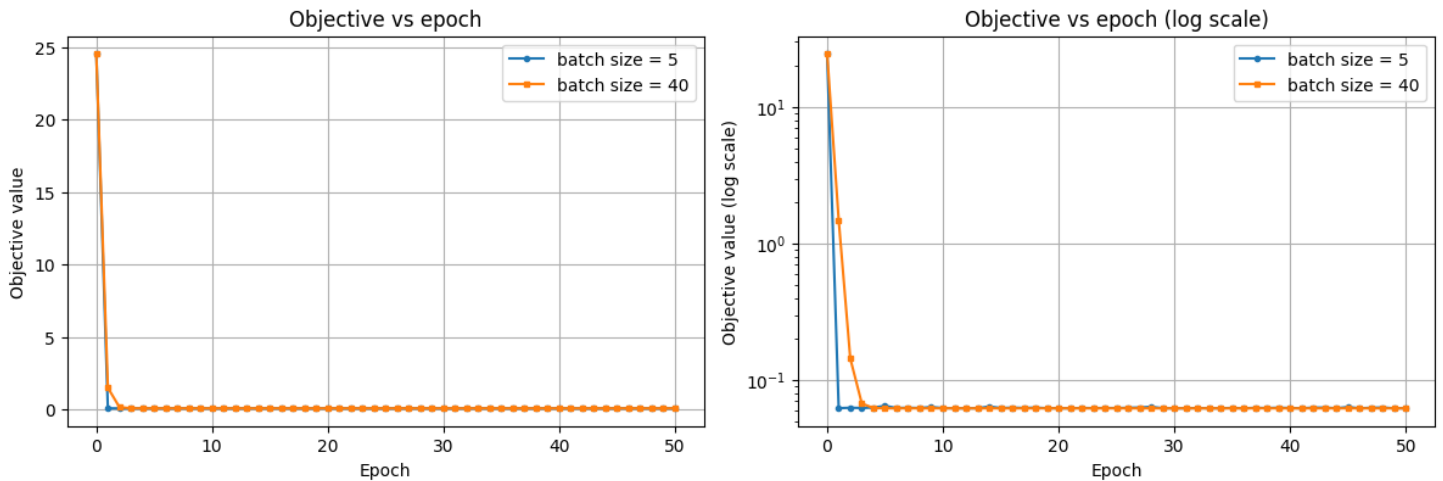
        for start in range(0, m, batch_size):
            end = min(start + batch_size, m)
            Xb = X_shuf[start:end]
            yb = y_shuf[start:end]

            g = (Xb.T @ (Xb @ x - yb)) / Xb.shape[0]
            x = x - alpha * g

        xs.append(x.copy())
        vals.append(f_A(x, X, y))

    return np.array(xs), np.array(vals)
```

Benchmark A: SGD Batch-Size Comparison



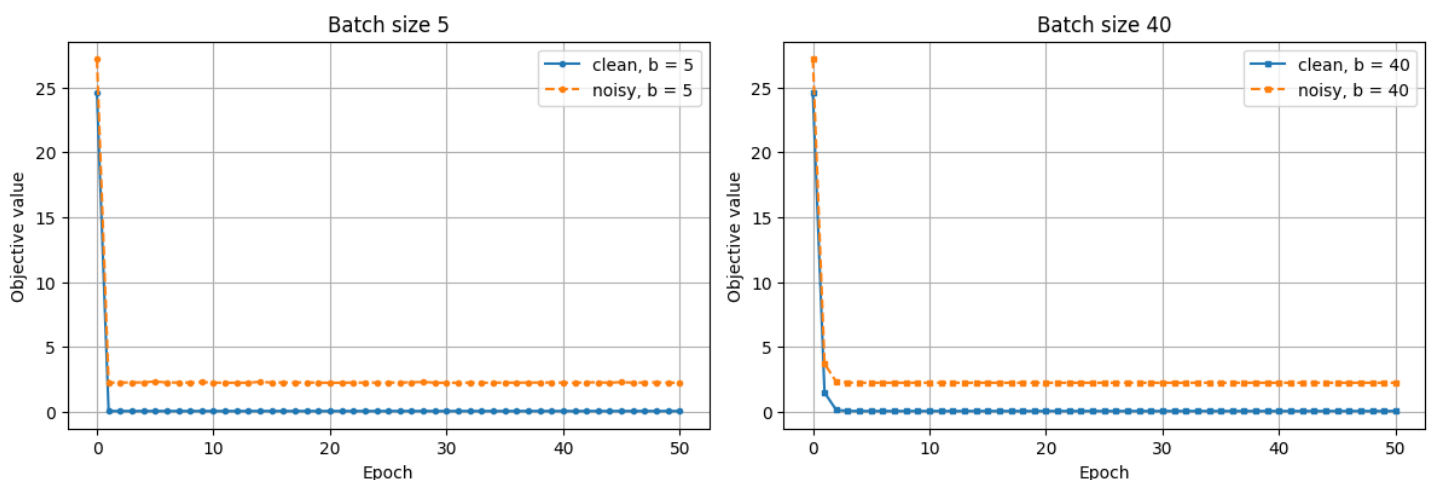
For the clean-data SGD experiment, both batch sizes converged rapidly. With batch size $b = 5$, the objective dropped from 24.555 to 0.0623 after the first epoch and ended at 0.0622 after 50 epochs. With batch size $b = 40$, the objective decreased more gradually at first, taking values 24.555, 1.485, 0.144, 0.0673, 0.0626, and then ending at 0.0622.

The smaller batch size therefore gave a faster initial reduction in the objective but also produced more variability in the later epochs. This is visible in the final few values, which fluctuate around 0.062 and 0.063. The larger batch size was smoother and more stable, with final values close to 0.0621 throughout. Thus, the batch-size comparison shows the standard trade-off: small batches give noisier but faster-looking progress, while larger batches give smoother convergence.

(e) Noisy SGD - How additional noise changes convergence

To study the effect of increased output noise, the SGD experiment on Benchmark A was repeated after multiplying the output noise level by an additional factor of 6.0, as required in the assignment.

Benchmark A: Effect of Additional Noise on SGD



Under the higher-noise setting, both SGD runs converged to worse objective values than in the clean-data case. With batch size $b = 5$, the objective started at 27.170 and fell to about 2.243 after the first epoch, ending at 2.241 after 50 epochs. With batch size $b = 40$, the objective started at the same value and decreased to 3.705, 2.318, 2.245, and 2.240 in the first epochs, ending at 2.238.

Compared with the clean-data runs, the effect of additional noise is clear. The final objective increased from about 0.062 to about 2.24, so the regression problem became harder to fit accurately. The smaller batch size again showed more variability in the final epochs, while the larger batch size was smoother and slightly better in final objective value. Therefore, increased output noise reduces the attainable accuracy of SGD and makes the stochastic fluctuations more important.

Q3:

For this question, the aim was to study Newton's Method and local approximation. The question first considered the one-dimensional test function $g(x) = x^4$, to compare its first order and second-order local approximations around a fixed point. After that, Newton's Method was applied to Benchmarks A, B, and C, and compared with a gradient descent baseline. Gradient descent was run for 80 iterations, while Newton's Method was run for 20 iterations with Hessian damping 10^{-8} , using the parameter settings specified in the assignment.

(a) Plotting the 1D function with 1st & 2nd-order approximations

The one-dimensional test function was

$$g(x) = x^4,$$

and the local approximations were constructed around the point $x_0 = 0.25$. The first derivative is:

$$g'(x) = 4x^3,$$

and the second derivative is:

$$g''(x) = 12x^2.$$

Therefore at $x_0 = 0.25$, these values are:

$$g(x_0) = 0.0039, \quad g'(x_0) = 0.0625, \quad g''(x_0) = 0.7500.$$

The first-order approximation was:

$$g(x) \approx g(x_0) + g'(x_0)(x - x_0),$$

while the second-order approximation was:

$$g(x) \approx g(x_0) + g'(x_0)(x - x_0) + \frac{1}{2}g''(x_0)(x - x_0)^2.$$

To generate the plot, these functions were implemented as:

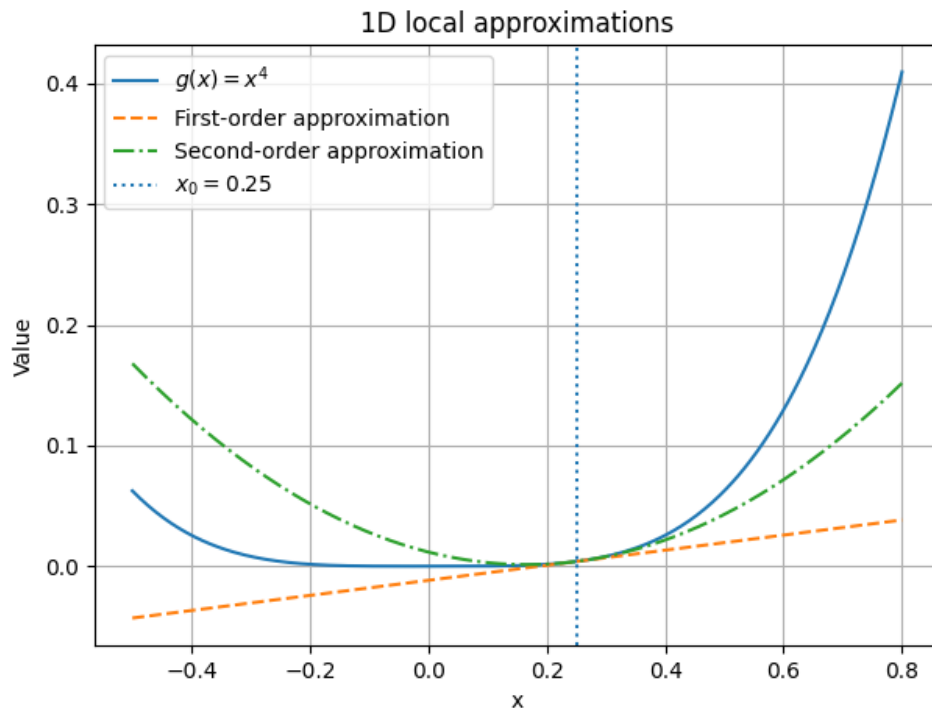
```
def g_1d(x):  
    return x**4  
  
def g1_1d(x):  
    return 4.0 * x**3  
  
def g2_1d(x):  
    return 12.0 * x**2  
  
def g_first_order(x, x0=0.25):
```

```

    return g_1d(x0) + g1_1d(x0) * (x - x0)

def g_second_order(x, x0=0.25):
    return g_1d(x0) + g1_1d(x0) * (x - x0) + 0.5 * g2_1d(x0) * (x - x0)**2

```



The plot of the three curves shows that the first-order approximation is only accurate close to x_0 , since it is just a tangent line. The second-order approximation captures the local curvature better and therefore remains closer to the original function in a wider neighbourhood around x_0 . This illustrates why second-order information can provide more useful local guidance than first-order information alone.

Using the second-order approximation, the one-dimensional Newton update is:

$$x_{k+1} = x_k - \frac{g'(x_k)}{g''(x_k)}.$$

At $x_0 = 0.25$, this gives

$$x_1 = 0.25 - \frac{0.0625}{0.75} = 0.1667.$$

This was also confirmed numerically in the code.

```

def newton_update_1d(x):
    return x - g1_1d(x) / g2_1d(x)

(...)
x0_1d = 0.25
x_grid_1d = np.linspace(-0.5, 0.8, 500)
g_vals_1d = g_1d(x_grid_1d)
g_lin_1d = g_first_order(x_grid_1d, x0=x0_1d)
g_quad_1d = g_second_order(x_grid_1d, x0=x0_1d)
x1d_newton_next = newton_update_1d(x0_1d)

```

(b) Implementing Newton's Method

Newton's Method was implemented using the update

$$x^{(t+1)} = x^{(t)} - \alpha (\nabla^2 f(x^{(t)}) + \lambda I)^{-1} \nabla f(x^{(t)}),$$

where $\lambda = 10^{-8}$ was the Hessian damping term. This was coded as:

```
def newton_method(f, grad_f, hess_f, x0, alpha=1.0, damping=1e-8, n_iters=20):
    x = x0.copy()

    xs = [x.copy()]
    vals = [f(x)]
    update_norms = []

    for _ in range(n_iters):
        g = grad_f(x)
        H = hess_f(x)

        H_damped = H + damping * np.eye(len(x))
        step = np.linalg.solve(H_damped, g)

        x = x - alpha * step

        xs.append(x.copy())
        vals.append(f(x))
        update_norms.append(np.linalg.norm(alpha * step))

    return np.array(xs), np.array(vals), np.array(update_norms)
```

For comparison, gradient descent was also implemented in a form that stores the update magnitudes:

```
def gradient_descent_with_updates(f, grad_f, x0, alpha, n_iters=80):
    x = x0.copy()

    xs = [x.copy()]
    vals = [f(x)]
    update_norms = []

    for _ in range(n_iters):
        g = grad_f(x)
        step = alpha * g
        x = x - step

        xs.append(x.copy())
        vals.append(f(x))
        update_norms.append(np.linalg.norm(step))

    return np.array(xs), np.array(vals), np.array(update_norms)
```

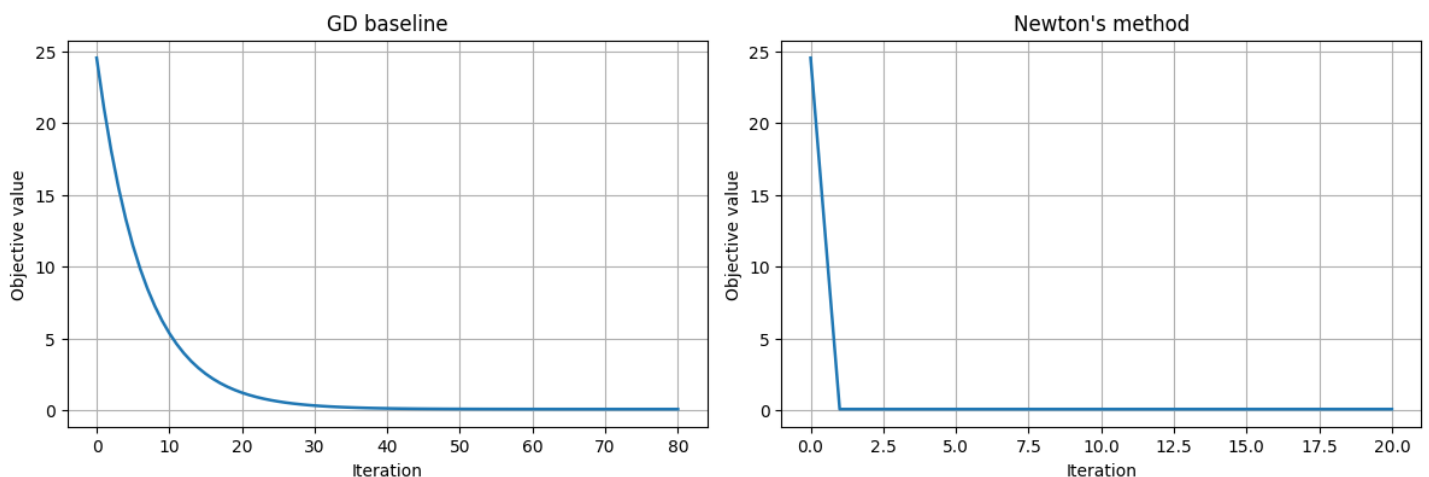
For Benchmark A, the Hessian is constant, so Newton's Method is especially effective. Starting from (-4, 6), gradient descent reduced the objective from 24.555 to 0.062 after 80 iterations, while Newton's Method reduced it to 0.062 in essentially a single step. The Newton iterates jumped immediately to about (3.011, 3.992), which is already the final solution to numerical precision. This is what would be expected for a quadratic objective.

For Benchmark B, gradient descent reduced the objective from 36.503 to 0.724, ending at approximately (0.582, 2.0). Newton's Method also converged to the same final objective value 0.724, but did so faster, reaching objective values 1.816, 0.769, 0.726, and 0.725 within the first few iterations. This shows that Newton's Method is again more efficient on this smooth low-dimensional problem.

For Benchmark C, the difference is less extreme. Gradient descent reduced the Rosenbrock objective from 24.20 to 3.868, while Newton's Method reduced it to 1.982. Thus, Newton's Method still performed better but did not solve the problem nearly as quickly or cleanly as in the first two benchmarks. This reflects the more difficult geometry of the Rosenbrock function.

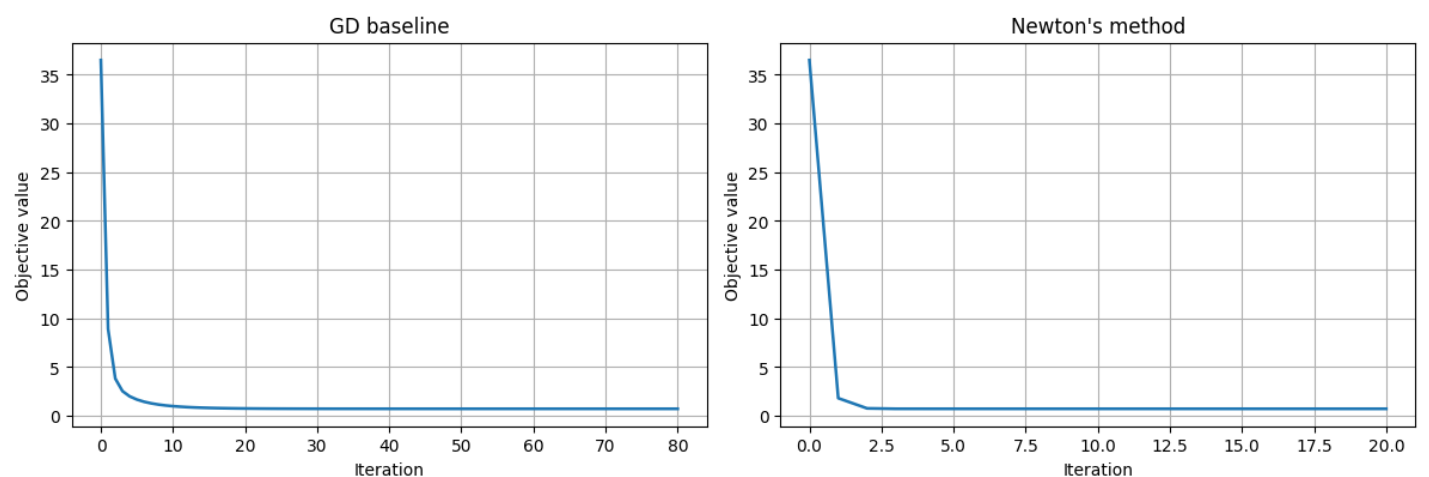
(c) Comparing Newton's Method with Gradient Descent

Benchmark A: Objective vs Iteration



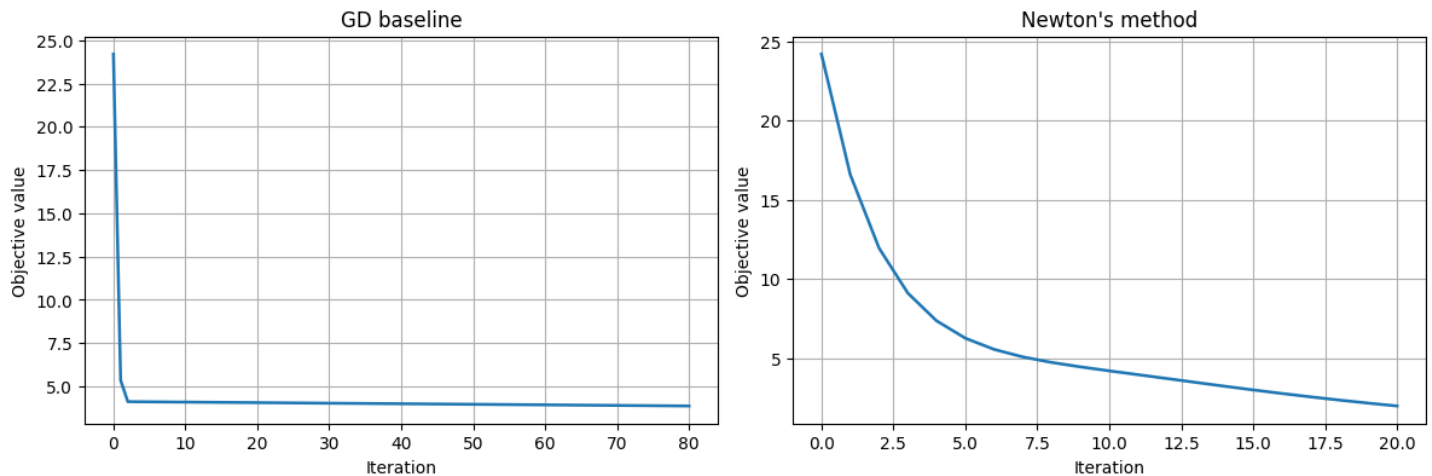
The objective-value plots show a clear advantage for Newton's Method on Benchmarks A and B. On Benchmark A, gradient descent decreased the objective gradually, with early values 24.555, 21.077, 18.094, 15.534, whereas Newton's Method dropped directly from 24.555 to 0.062 after one iteration and stayed there. Thus, on the quadratic regression loss, Newton's Method converged essentially immediately.

Benchmark B: Objective vs Iteration



On Benchmark B, gradient descent again converged steadily, with initial objective values 36.503, 8.925, 3.827, 2.537, 1.998. Newton's Method was faster, reducing the objective to 1.816 after one step and then to 0.769, 0.726, and 0.725 over the next few iterations. By the end, both methods reached the same final value 0.724, but Newton's Method required far fewer steps.

Benchmark C: Objective vs Iteration

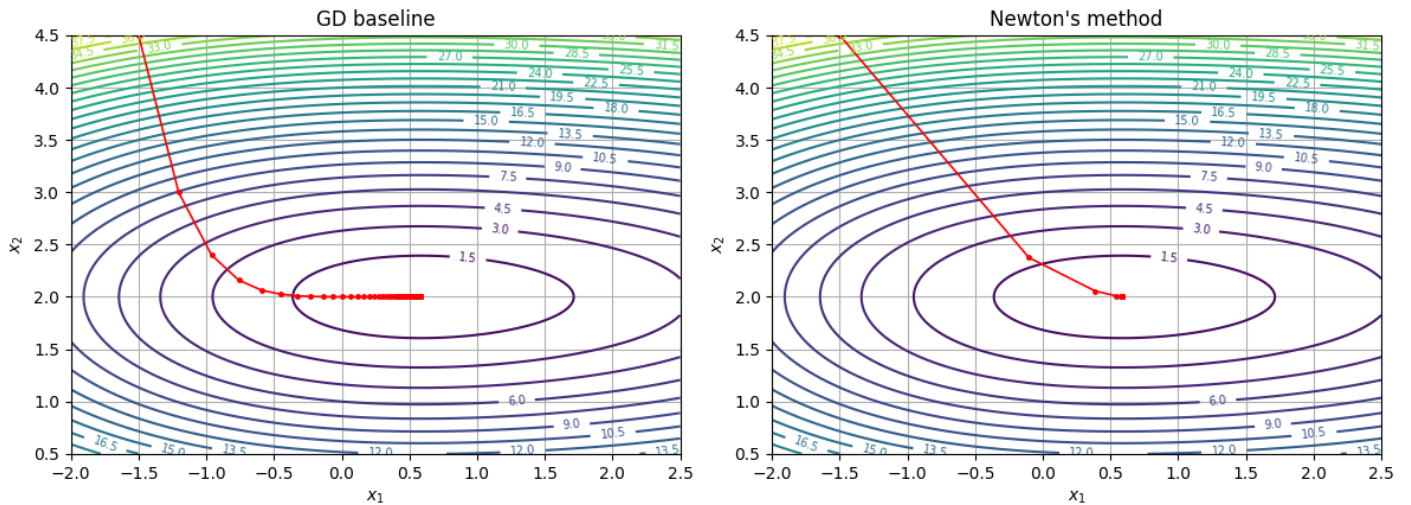


On Benchmark C, Newton's Method still improved over gradient descent, but the difference was less dramatic. Gradient descent reduced the objective to 3.868, while Newton's Method reached 1.982 after 20 iterations. Thus, Newton's Method was still better in terms of final objective value, but it did not display the same near-instant convergence seen in Benchmarks A and B. The Rosenbrock function is more difficult because the curvature changes strongly across the domain, so the Hessian information is less straightforward to exploit.

The comparison shows that Newton's Method performs best when the objective is well approximated locally by a quadratic model. This is exactly the case for Benchmark A, and approximately the case for Benchmark B near the minimiser. For Rosenbrock, second-order information still helps, but the landscape is difficult enough that the improvement is more moderate.

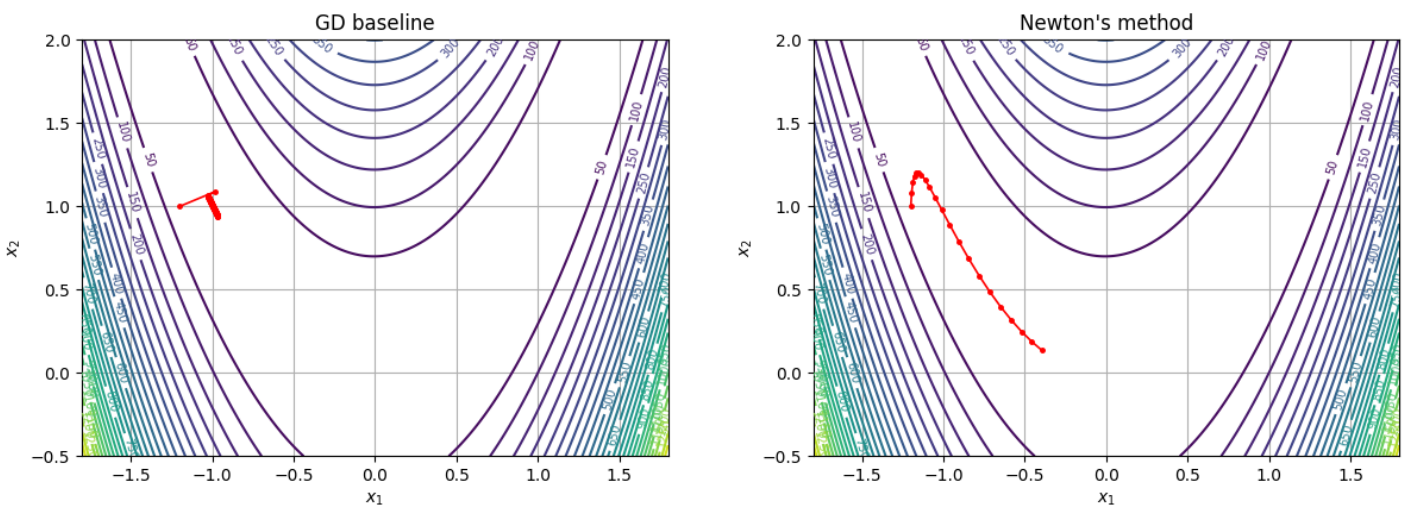
(d) Contour Trajectories for Benchmark B & C

Benchmark B: Contour & Optimisation Trajectories



For Benchmark B, the contour plots show that both methods move toward the same minimiser near (0.582, 2). The gradient descent trajectory is smooth and gradual, while Newton's Method moves more directly and reaches the low-objective region in only a few steps. This matches the numerical results, where Newton's Method rapidly reduced the objective and then stabilised.

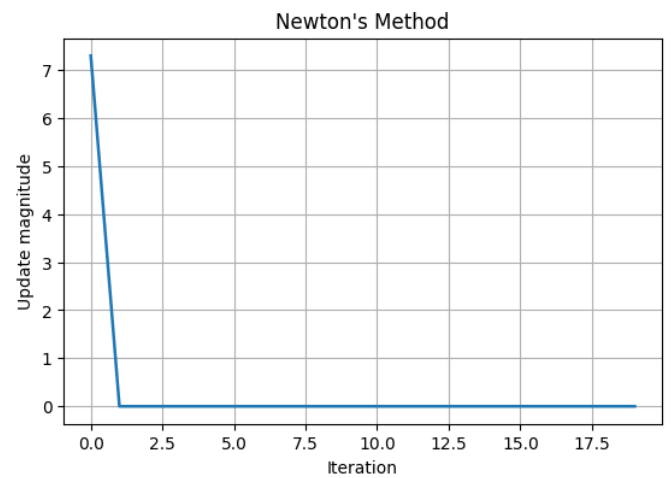
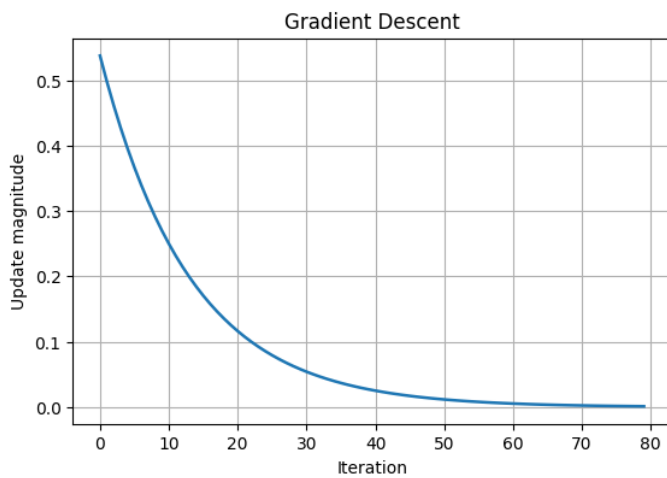
Benchmark C: Contour & Optimisation Trajectories



For Benchmark C, the contour plots show a larger difference in the paths. Gradient descent follows the Rosenbrock valley slowly and remains relatively far from the minimum after 80 iterations. Newton's Method makes more direct progress and reaches a lower-objective region, but its path still reflects the difficulty of the curved valley. Unlike the previous benchmarks, Newton's trajectory cannot jump immediately to the solution. Instead, it continues to make gradual improvement throughout the run. Therefore, the contour plots confirm that Newton's Method is more effective than gradient descent here, but not to that great of a degree.

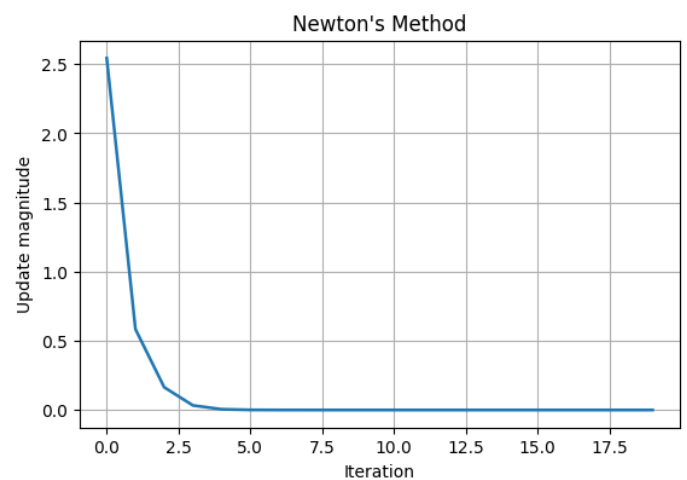
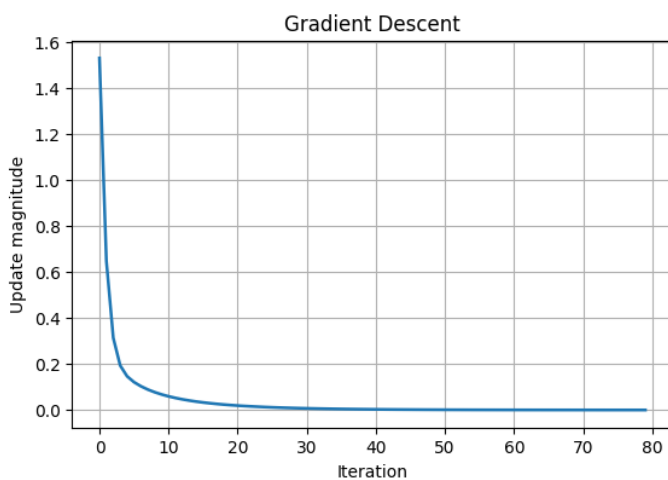
(e) Update Magnitude vs Iteration

Benchmark A: Update Magnitude vs Iteration



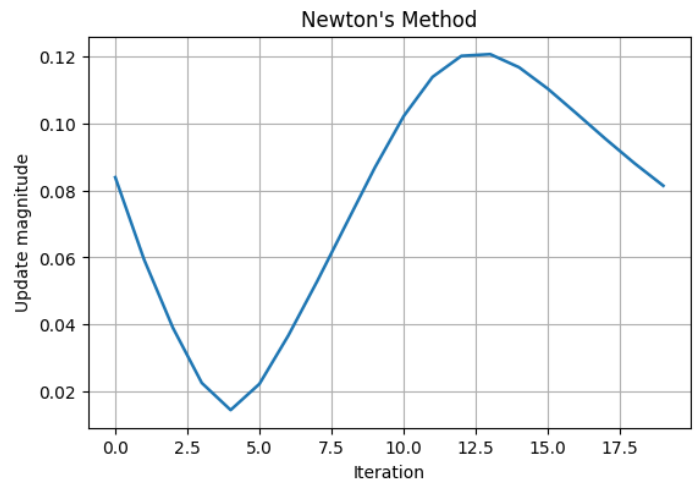
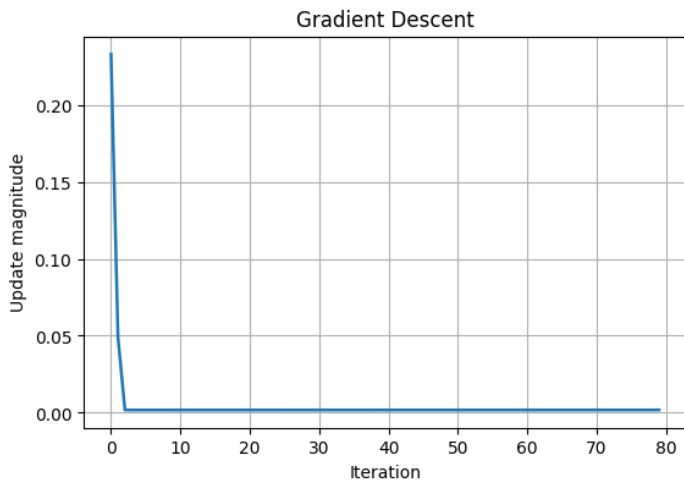
For Benchmark A, gradient descent had initial update magnitudes 0.537, 0.498, 0.461, 0.427, decreasing gradually to about 0.0013 by the end. Newton's Method, by contrast, had a first update magnitude of 7.293, and all later updates were essentially zero. This is consistent with the method reaching the optimum in one Newton step.

Benchmark B: Update Magnitude vs Iteration



For Benchmark B, gradient descent again showed gradual decay in the update sizes, from 1.529 initially down to values near 0.0001. Newton's Method had update magnitudes 2.544, 0.584, 0.164, 0.033, 0.005 in the first few steps, and then effectively zero afterwards. This shows that Newton's Method took a few large corrective steps and then settled rapidly.

Benchmark C: Update Magnitude vs Iteration



For Benchmark C, the update magnitudes show a different pattern. Gradient descent had a first update magnitude of 0.233, then smaller updates near 0.0018, which remained almost constant. Newton's Method started with smaller updates such as 0.084, 0.059, 0.039, 0.023, 0.015, but unlike the earlier benchmarks, its update magnitudes did not shrink toward zero. In fact, the last few update magnitudes were still around 0.110, 0.103, 0.095, 0.088, 0.081. This shows that Newton's Method was still making substantial corrections on Rosenbrock even at the end of the 20 iterations, which is consistent with the fact that it had not yet converged fully.

From this it can be surmised that Newton's Method converges quickly on easier problems, whereas on Rosenbrock it continues to make nontrivial steps because the geometry remains challenging.

Q4:

For this question, the aim was to study derivative approximation and derivative-free optimisation. In Part A, Benchmark B was used to compare exact gradient descent, finite-difference gradient descent, and Nesterov random search. In Part B, Benchmark C was used to study two derivative-free methods, namely the Nelder-Mead simplex method and brute-force grid search.

(a) Methods Comparison (on Benchmark B)

The finite-difference gradient approximation was implemented using the forward-difference formula

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + \delta e_i) - f(x)}{\delta},$$

and this was coded as:

```
def grad_finite_diff(f, x, delta):
    n = len(x)
    g = np.zeros(n)
    fx = f(x)

    for i in range(n):
        e = np.zeros(n)
        e[i] = 1.0
        g[i] = (f(x + delta * e) - fx) / delta

    return g
```

Using this approximation, finite-difference gradient descent was implemented with the update

$$x^{(t+1)} = x^{(t)} - \alpha \tilde{\nabla} f(x^{(t)}),$$

where $\tilde{\nabla} f$ is the approximate gradient:

```
def finite_diff_gd(f, x0, alpha, delta, n_iters=120):
    x = x0.copy()
    xs = [x.copy()]
    vals = [f(x)]

    for _ in range(n_iters):
        g = grad_finite_diff(f, x, delta)
        x = x - alpha * g
        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(vals)
```

Nesterov random search was implemented using the update

$$x_{k+1} = x_k - \alpha \frac{f(x_k + \delta u_k) - f(x_k)}{\delta} u_k,$$

where u_k is a random unit vector. This was implemented as the function:

```
def nesterov_random_search(f, x0, alpha, delta, n_iters=220, seed=123):
    rng = np.random.default_rng(seed)

    x = x0.copy()
    xs = [x.copy()]
    vals = [f(x)]

    for _ in range(n_iters):
        u = rng.normal(size=len(x))
        u = u / np.linalg.norm(u)

        directional_est = (f(x + delta * u) - f(x)) / delta
        x = x - alpha * directional_est * u

        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(vals)
```

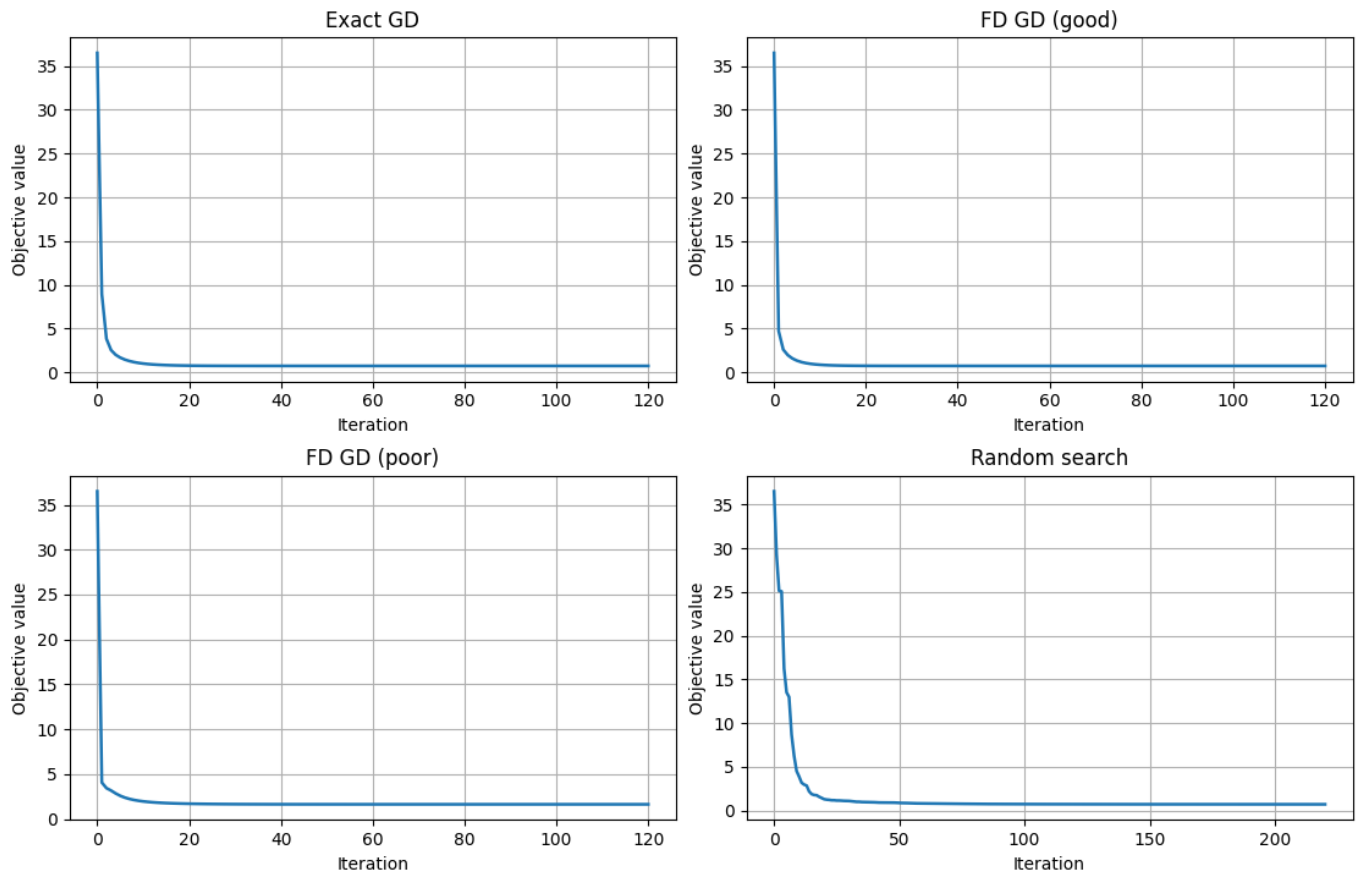
The benchmark comparison shows a clear performance ranking. Exact gradient descent performed best, reducing the objective from 36.503 to 0.724, and converging to approximately (0.582, 2.0). Finite-difference GD with the good setting $\delta = 0.05$ also worked well, reaching final objective 0.728 at approximately (0.558, 1.975). This is slightly worse than exact GD, but still close.

Finite-difference GD with the poor setting $\delta = 0.8$ performed worse, ending at (0.198, 1.6) with final objective 1.64. This shows that a poor finite-difference step can produce a poor gradient approximation and therefore weaker optimisation performance.

Nesterov random search performed surprisingly well overall. Starting from the same initial point, it reached approximately (0.580, 2.006), with final objective 0.725. This is extremely close to the exact gradient descent result, although the random-search trajectory was noisier. Therefore, on this benchmark, random search was competitive in final accuracy, but less smooth in its path.

(b) Objective Value vs Iteration

Benchmark B: Objective vs Iteration



Exact gradient descent produced the cleanest descent curve. Its objective fell from 36.503 to 1.998 in its initial iterations, before settling at 0.7244.

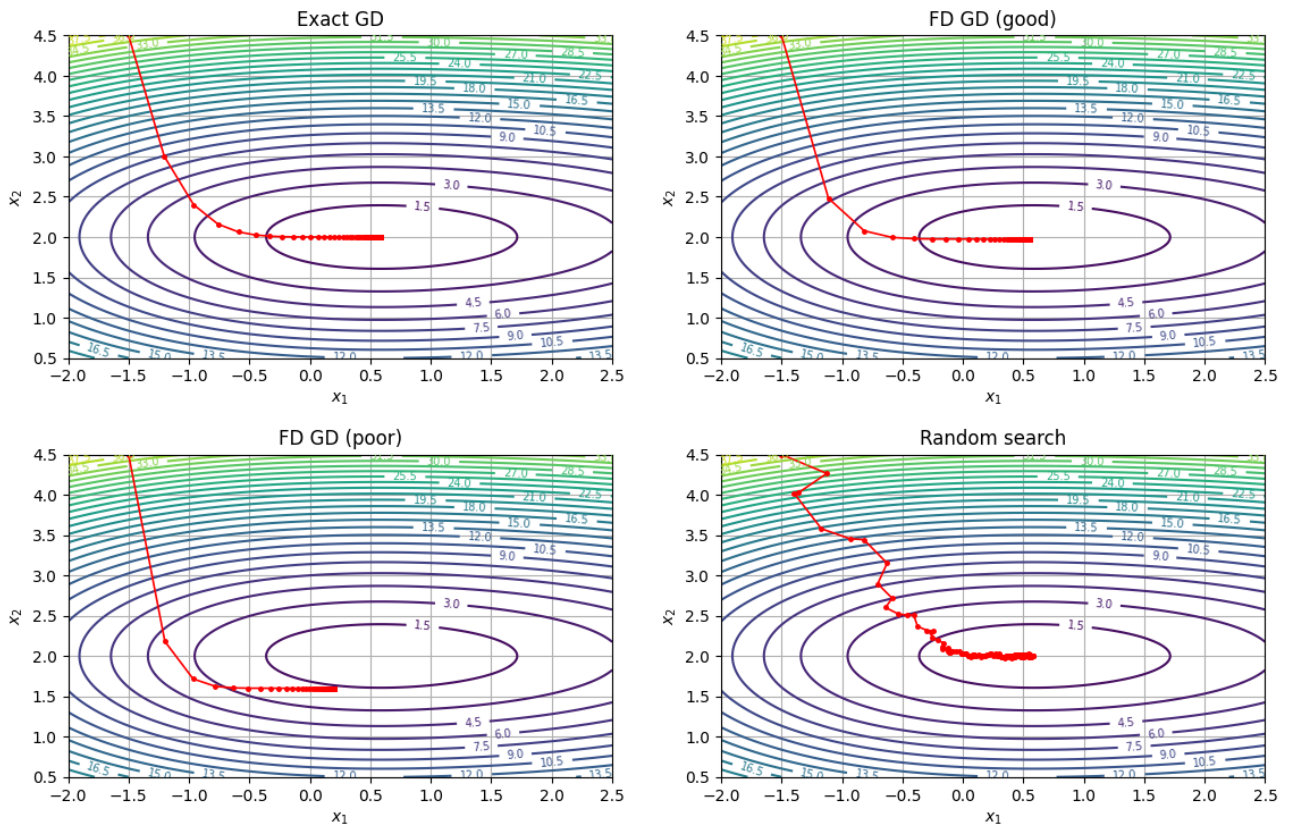
Finite-difference GD with the good setting $\delta = 0.05$ also decreased the objective quickly, with its first values 36.503, 4.715, 2.595, 1.960, 1.579. It converged to 0.728, which is slightly worse than exact GD, but still shows that the approximation was sufficiently accurate for the method to work well.

Finite-difference GD with the poor setting $\delta = 0.8$ decreased the objective at first, with values 36.503, 4.067, 3.445, 3.178, 2.855, but then stalled further from the optimum, finishing at 1.6402. The larger δ therefore made the approximate gradient too inaccurate for precise convergence.

Nesterov random search showed the noisiest objective curve. Its first few values were 36.503, 29.242, 25.130, 25.066, 16.234, showing less regular progress than the gradient-based methods. However, by the end it still reached 0.7246, which is close to the best value. Thus, the objective plot shows that random search was less smooth, but still effective in final performance on this low-dimensional problem.

(c) Contour Trajectories for Benchmark B

Benchmark B: Contour & Optimisation Trajectories



Exact gradient descent followed the most direct and stable path to the minimiser. The trajectory moved steadily toward the basin around $(0.582, 2)$, with no visible irregularity.

Finite-difference GD with the good setting followed a path like exact GD, but it converged to a point slightly offset from the true minimiser. This reflects the approximation error introduced by the finite-difference gradient.

Finite-difference GD with the poor setting followed a less accurate path and settled well away from the optimum. The large finite-difference step made the estimated gradient too crude, which distorted the descent direction.

Nesterov random search produced the most irregular trajectory, since its search direction depends on random unit vectors rather than exact coordinate gradients. Even so, the path still moved into the correct low-objective region and ended close to the minimiser. This shows that derivative-free random search can still work well in low-dimensional settings, though with noisier behaviour than gradient-based methods.

(d) Nelder-Mead Trajectory on Benchmark C

For Benchmark C, the Nelder–Mead simplex method was implemented using a standard reflection-expansion-contraction-shrink scheme on a 2D simplex. The code began with an initial simplex formed from the starting point and two coordinate wise step offsets:

```
def nelder_mead_2d(f, x0, step=0.35, n_iters=160, alpha=1.0, gamma=2.0, rho=0.5, sigma=0.5):
    x0 = x0.copy()

    simplex = np.array([
        x0,
        x0 + np.array([step, 0.0]),
        x0 + np.array([0.0, step])
    ])

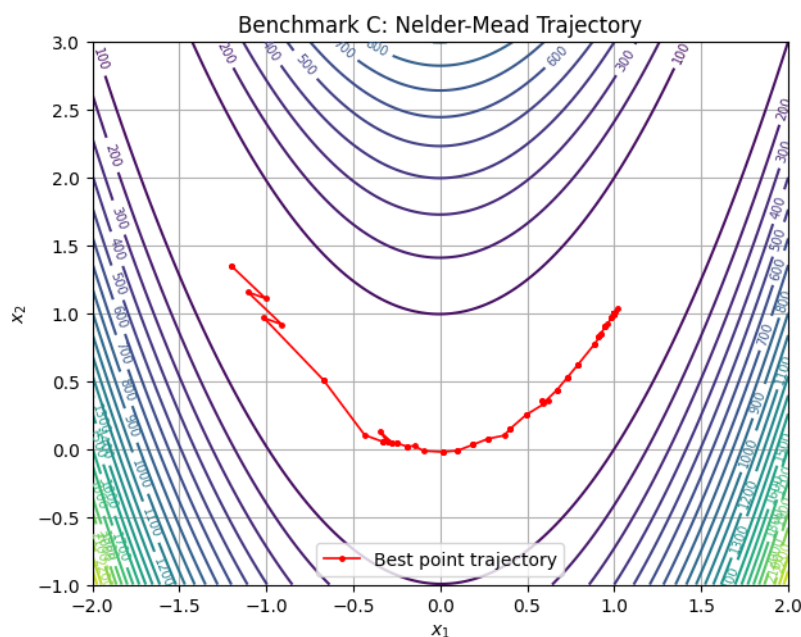
    traj_best = []
    vals_best = []
    simplex_hist = []

    for _ in range(n_iters):
        fvals = np.array([f(p) for p in simplex])
        idx = np.argsort(fvals)

        simplex = simplex[idx]
        fvals = fvals[idx]

        best = simplex[0].copy()
        worst = simplex[-1].copy()
        ...
```

Numerically, Nelder–Mead performed extremely well on the Rosenbrock function. The best-point trajectory began at approximately $(-1.2, 1.35)$, with initial objective 5.650, and converged to $(1.0, 1.0)$ with final objective essentially 0.0. The initial objective values were 5.650, 5.650, 5.076, 4.888, 4.451, after which the method continued improving until it reached the exact minimiser numerically.



The contour trajectory plot shows that the simplex-based search was able to move through the Rosenbrock valley and eventually converge to the global minimum at (1,1). This is a strong result for a derivative-free method, especially on a benchmark that is often difficult for simple first-order methods.

(e) Grid-Search Sampling Pattern & Best-So-Far Value on Benchmark C

The brute-force grid search was implemented by sampling a 55×55 grid over the region

$$x_1 \in [-2, 2], \quad x_2 \in [-1, 3],$$

and recording both the sampled objective values and the best-so-far value throughout the search:

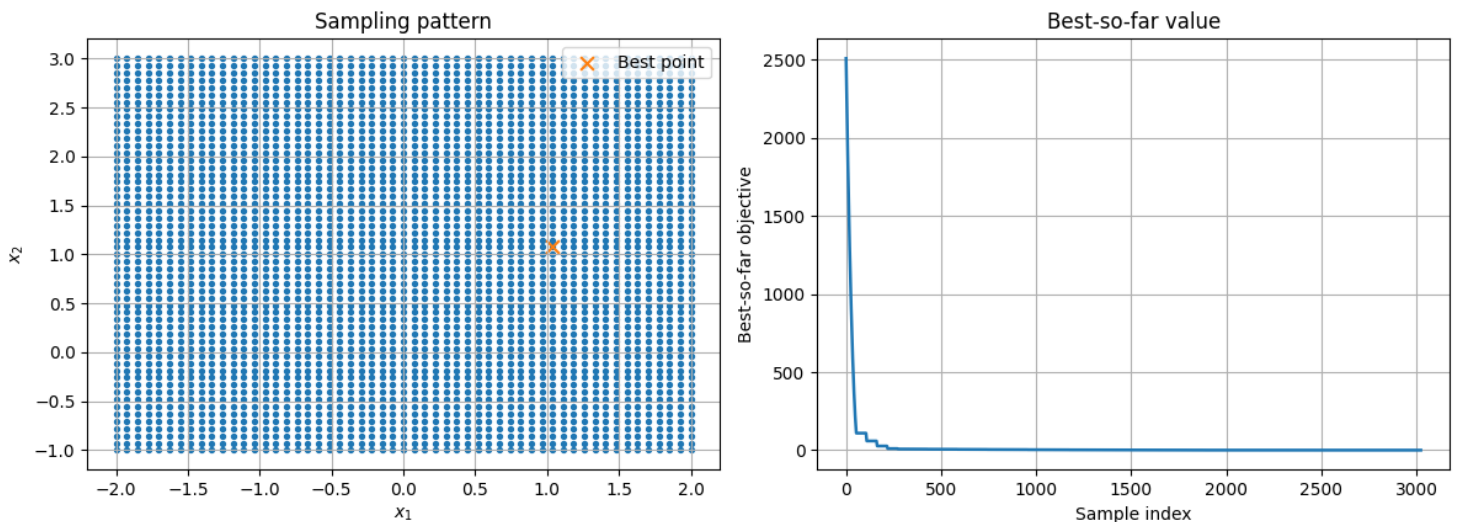
```
def grid_search_2d(f, x1_range=(-2, 2), x2_range=(-1, 3), grid_n=55):
    x1_vals = np.linspace(x1_range[0], x1_range[1], grid_n)
    x2_vals = np.linspace(x2_range[0], x2_range[1], grid_n)

    sampled_points = []
    sampled_vals = []
    best_so_far = []

    best_val = np.inf
    best_point = None

    for x1 in x1_vals:
        for x2 in x2_vals:
            p = np.array([x1, x2])
            val = f(p)
            (...)
```

Benchmark C: Grid Search Results



The grid search found a best point of approximately (1.037, 1.074), with objective value 0.0016. This is close to the true Rosenbrock minimiser (1,1), but not exact because the method is limited by the resolution of the grid.

The best-so-far value plot shows a gradual improvement as more points are sampled. The initial best-so-far values were extremely large, around 2509.0, 2435.5, 2363.0, 2291.7, 2221.5, and the final best-so-far value flattened at 0.0016. This demonstrates the brute-force nature of grid search, that being, it is a reliable method, but computationally inefficient, as it evaluates many poor points before reaching the good region.

The sampling-pattern plot also makes this clear. The method evaluates the entire grid uniformly rather than adapting to promising regions, so its accuracy depends directly on the grid density.

(f) Comparing computational behaviour and accuracy

Across the methods in Part A, exact gradient descent was the most accurate and smoothest method on Benchmark B. Finite-difference GD with a good step size was close behind, showing that numerical gradient approximation can work well when δ is chosen carefully. Finite-difference GD with a poor step size performed worse, showing that the method is highly sensitive to the approximation parameter. Nesterov random search achieved a final objective value close to exact GD, but its behaviour was noisier and less efficient in the early stages.

Across the methods in Part B, Nelder–Mead was the most successful derivative-free method on Benchmark C. It converged all the way to (1,1) with essentially zero final objective value, despite using no gradient information. Grid search also located a point close to the minimiser, but only to the resolution of the grid, giving final value 0.0016 rather than exact convergence.

In terms of computational behaviour, gradient-based methods were clearly more efficient on Benchmark B when exact gradients were available. Finite differences added approximation error, and random search added stochastic noise. On Benchmark C, Nelder–Mead showed that an adaptive derivative-free method can still be effective, while grid search was the simplest but also the most brute-force and resolution-limited method.

The results show that exact derivatives remain the most efficient choice when they are available, but well-designed derivative-free methods can still perform strongly, especially in low-dimensional problems. Finite-difference methods occupy a middle ground where they can work well, but only if the approximation parameter is chosen appropriately.

Q5:

For this question, the aim was to investigate constrained optimisation on Benchmark B under the constraint

$$x_1 \geq 0.5.$$

Three approaches were compared, unconstrained gradient descent, projected gradient descent, and penalty-based gradient descent with three different penalty weights. The benchmark function was the same nonlinear objective used earlier,

$$f(x_1, x_2) = (x_1 - 1)^2 + 5(x_2 - 2)^2 + \sin(x_1),$$

but now the iterates had to respect the additional feasible-set condition. The initial point was $x^{(0)} = (0.2, 4.0)$, which is infeasible because $x_1 < 0.5$. All methods were implemented in Python using NumPy and Matplotlib, and each run was carried out for 100 iterations.

(a) Projected & Penalty-Based Gradient Descent

The feasible set for this question is defined as:

$$X = \{x \in \mathbb{R}^2: x_1 \geq 0.5\}.$$

For projected gradient descent, the projection is simple, only the first coordinate may need to be clipped upward to 0.5. This was implemented as:

```
def proj_x1_ge_05(x):  
    y = x.copy()  
    y[0] = max(0.5, y[0])  
    return y
```

The projected gradient descent update was then

$$x^{(t+1)} = \Pi_X \left(x^{(t)} - \alpha \nabla f(x^{(t)}) \right),$$

which was coded as:

```
def projected_gradient_descent(f, grad_f, x0, alpha, proj, n_iters=100):  
    x = x0.copy()  
    xs = [x.copy()]  
    vals = [f(x)]  
  
    for _ in range(n_iters):  
        x = proj(x - alpha * grad_f(x))  
        xs.append(x.copy())  
        vals.append(f(x))  
  
    return np.array(xs), np.array(vals)
```

For the penalty method, the objective was modified to

$$F_{\lambda}(x) = f(x) + \lambda \max(0, 0.5 - x_1),$$

so, the penalty term is active only when the constraint is violated. The corresponding functions were implemented as:

```
def penalty_q5(x, lam):  
    x1 = x[0]  
    return lam * max(0.0, 0.5 - x1)  
  
def F_q5(x, lam):  
    return f_B(x) + penalty_q5(x, lam)
```

and for the sub gradient:

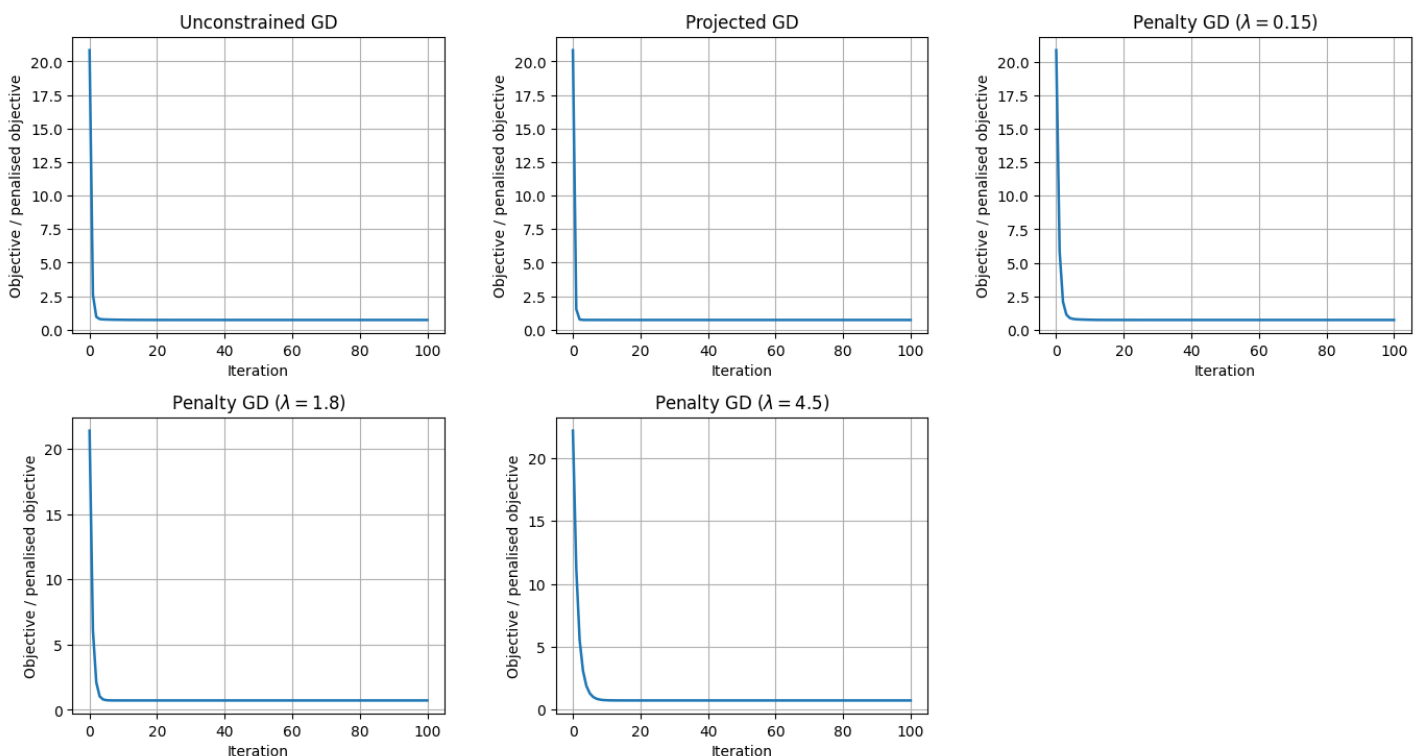
```
def subgrad_F_q5(x, lam):  
    g = grad_B(x).copy()  
  
    if x[0] < 0.5:  
        g += np.array([-lam, 0.0])  
  
    return g
```

The penalty gradient descent update was then applied for three values of λ , namely 0.15, 1.8, and 4.5, using the step sizes specified in the assignment. For comparison, an unconstrained gradient descent baseline was also run, even though it does not explicitly enforce the feasible set.

(b) Objective or Penalised Objective vs Iteration

The objective-value plots show that all methods eventually reached essentially the same final value, but they did so in different ways.

Benchmark B with constraint: Objective / Penalised Objective vs Iteration



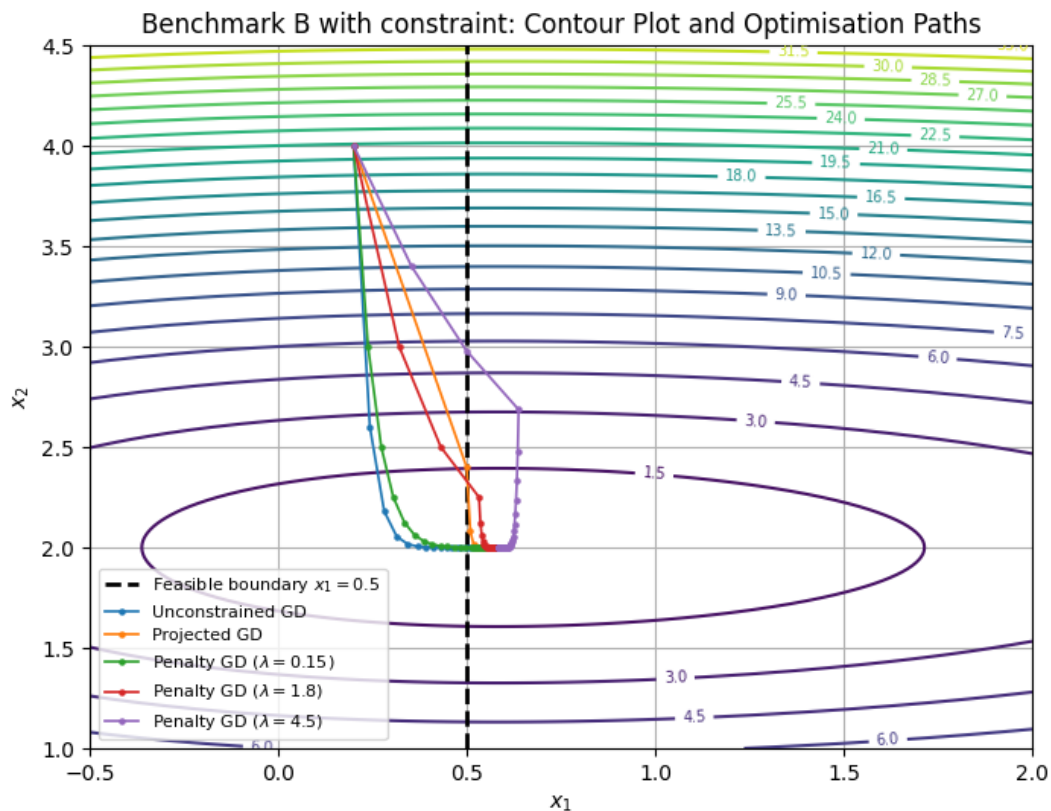
The unconstrained gradient descent baseline reduced the objective from 20.839 to 0.724, converging to approximately (0.582, 2.0). Its first few objective values were 20.839, 2.613, 0.956, 0.794, 0.769, showing a sharp decrease from the onset.

Projected gradient descent also converged to the same final objective 0.724, but it did so more efficiently in the initial iterations. Its first few values were 20.839, 1.529, 0.76, 0.729, 0.727, which are lower than those of the unconstrained baseline at the same early stages. This indicates that enforcing feasibility immediately was effective in this circumstance.

For the penalty methods, the plotted quantity was the penalised objective $F_\lambda(x)$. With $\lambda = 0.15$, the initial penalised objective was 20.884, which then decreased to 5.855, 2.082, 1.125, 0.874 and eventually to 0.724. With $\lambda = 1.8$, the initial value was 21.379, and the initial values were 6.099, 2.115, 1.039, 0.804, again eventually reaching 0.724. With $\lambda = 4.5$, the initial penalised objective was higher at 22.189, and its first few values were 11.223, 5.535, 3.08, 1.879, so the descent was noticeably slower in the early stage. Even so, it still converged to the same final value 0.724.

Thus, the objective plots show that all methods ultimately reached the same constrained minimiser numerically, but projected gradient descent produced the cleanest and fastest early descent, while larger penalty parameters made the early penalised objective substantially larger.

(c) **Contour Plot with Feasible Boundary & Optimisation Paths**



The unconstrained gradient descent trajectory begins from the infeasible point (0.2, 4.0) and gradually moves into the feasible region as the iterations progress. Since the unconstrained minimiser of Benchmark B already lies in the feasible set, the method eventually reaches the same constrained solution, even without explicitly enforcing feasibility.

Projected gradient descent behaves differently in the first step. Since it immediately projects onto the feasible boundary, the first iterate has $x_1 = 0.5$, and from then on, the path stays fully feasible. This gives a more direct route toward the constrained optimum. Numerically, the initial x_1 values were 0.2, 0.5, 0.510, 0.518, 0.526, showing the immediate correction onto the feasible set.

The penalty methods interpolate between these two behaviours. With $\lambda = 0.15$, the iterates remain infeasible for longer and approach feasibility more gradually. The first few x_1 values were 0.2, 0.239, 0.274, 0.306, 0.335. With $\lambda = 1.8$, the push toward feasibility is stronger, and the initial x_1 values were 0.2, 0.321, 0.432, 0.533, 0.537, so the method becomes feasible earlier. With $\lambda = 4.5$, the push is even stronger, with first few x_1 values 0.2, 0.354, 0.499, 0.638, 0.636. This means the method crosses the boundary aggressively and then moves back toward the final optimum.

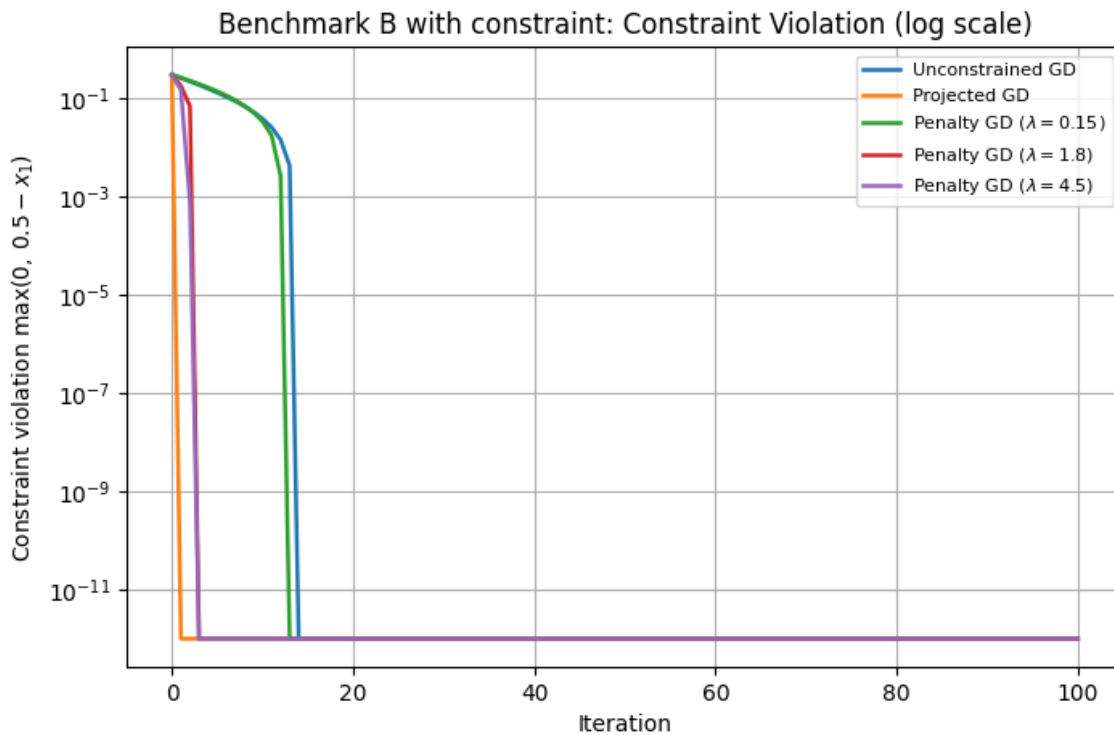
The contour plot therefore confirms the expected effect of the penalty parameter: small penalties allow longer violation of the constraint, while larger penalties enforce feasibility more aggressively.

(d) Constraint Violation (log scale)

The constraint violation was measured by:

$$\max(0, 0.5 - x_1),$$

which is zero whenever the iterate is feasible. This was plotted on a logarithmic scale so that the rate at which each method removes infeasibility could be compared more clearly.

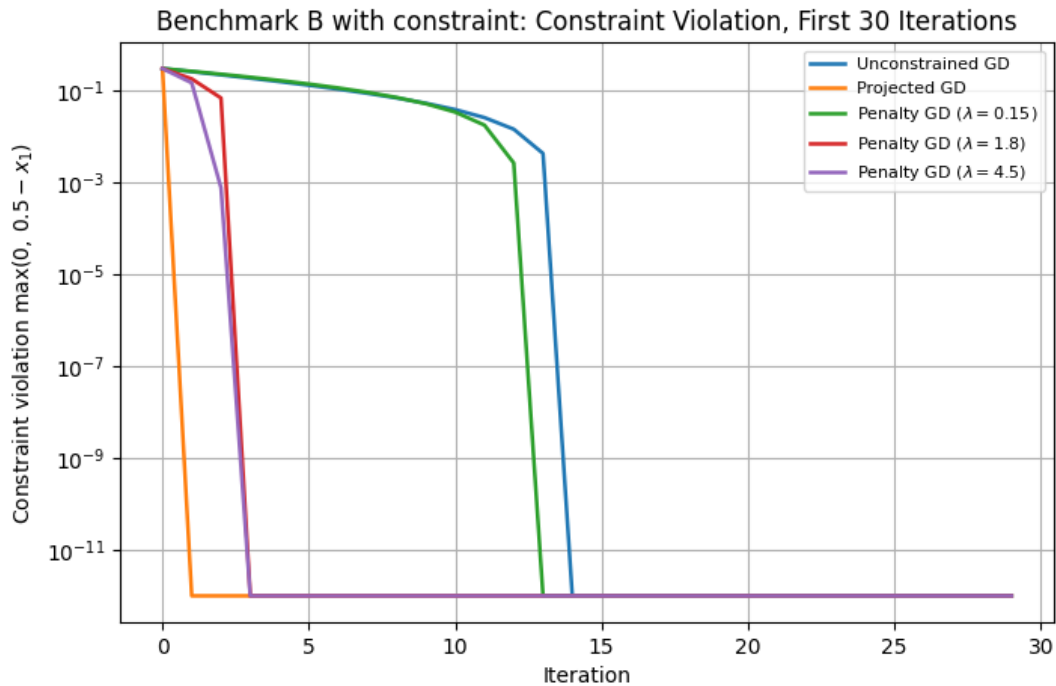


All five methods had a final constraint violation of exactly 0.00 indicating that every method eventually entered and remained in the feasible set. However, the rate at which this happened differed between methods. Projected gradient descent had the strongest feasibility behaviour because after the first projection step, the violation became zero immediately and stayed there. This is expected, since projection explicitly enforces the constraint at every iteration.

The unconstrained gradient descent method also reduced the violation to zero, but only indirectly through the natural movement of the objective toward its unconstrained minimiser. Since that minimiser lies in the feasible set, the violation eventually disappeared even without any constraint-handling mechanism.

For the penalty methods, the log-scale violation plot is the most informative. The larger the penalty parameter λ , the faster the violation decreased. The run with $\lambda = 0.15$ reduced infeasibility the most slowly, while $\lambda = 1.8$ and especially $\lambda = 4.5$ forced the iterate across the boundary earlier. Therefore, the violation plot clearly shows how the penalty weight controls the speed of feasibility enforcement.

(e) Constraint Violation (@ iterations 0-30)



Projected gradient descent achieves zero violation almost immediately, since the first step sends x_1 directly to 0.5. This makes it the fastest method in terms of feasibility restoration.

The unconstrained gradient descent method reduces the violation steadily, but more gradually, since it is not directly targeting the constraint. The penalty method with $\lambda = 0.15$ behaves somewhat similarly, because the penalty is too weak to dominate the optimisation early on.

By contrast, the penalty methods with $\lambda = 1.8$ and $\lambda = 4.5$ reduce the violation more rapidly. In particular, $\lambda = 4.5$ pushes the iterate into the feasible region early, although the stronger penalty also makes the early penalised objective larger, as seen in Part (b). This illustrates the trade-off in the penalty method: stronger penalties improve constraint enforcement but may distort the optimisation path more strongly.

(f) Method Comparison

All five methods converged to essentially the same constrained solution near (0.582, 2.0), with final objective value 0.724 and zero final constraint violation. This is because the constrained optimum for this problem lies in the same region as the unconstrained minimiser.

Even so, the methods behaved quite differently during the optimisation process. Projected gradient descent was the cleanest constrained method. It enforced feasibility immediately and produced the strongest early reduction in the objective. In this experiment, it was the most natural and effective way to handle the constraint.

The unconstrained gradient descent baseline also reached the feasible optimum, but only because the unconstrained solution happened to satisfy the constraint. In a more difficult constrained problem, this would not be guaranteed. Therefore, while it worked here, it is not a reliable general strategy for constrained optimisation.

The penalty methods all worked, but their behaviour depended strongly on the penalty weight. A small penalty such as $\lambda = 0.15$ allowed constraint violation to persist longer. Larger penalties such as $\lambda = 1.8$ and $\lambda = 4.5$ enforced feasibility more quickly but also produced larger early penalised objective values and more aggressive motion across the boundary.

Thus, the results show the main contrast between projection and penalty approaches. Projection gives immediate feasibility and simple behaviour when the projection is easy to compute. Penalty methods are more flexible in general, but they require careful tuning of λ , since too small a value enforces the constraint too weakly and too large a value can distort the optimisation path.

Q6:

This question investigates linear programming and the Frank–Wolfe algorithm over the box-constrained feasible set

$$X = \{x \in \mathbb{R}^2: 0.5 \leq x_1 \leq 5, -5 \leq x_2 \leq 10\}.$$

A linear programme is first solved over X , followed by the application of Frank–Wolfe to two quadratic objectives: one with an interior optimum and one with a boundary optimum.

(a) Linear Programme over X

The linear objective is:

$$f(x) = a^\top x, \quad a = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Since the feasible set is a rectangle, the minimum of a linear function over X must occur at one of the four vertices. These vertices were defined in python as:

```
vertices_X = np.array([
    [0.5, -5.0],
    [5.0, -5.0],
    [5.0, 10.0],
    [0.5, 10.0]
])
```

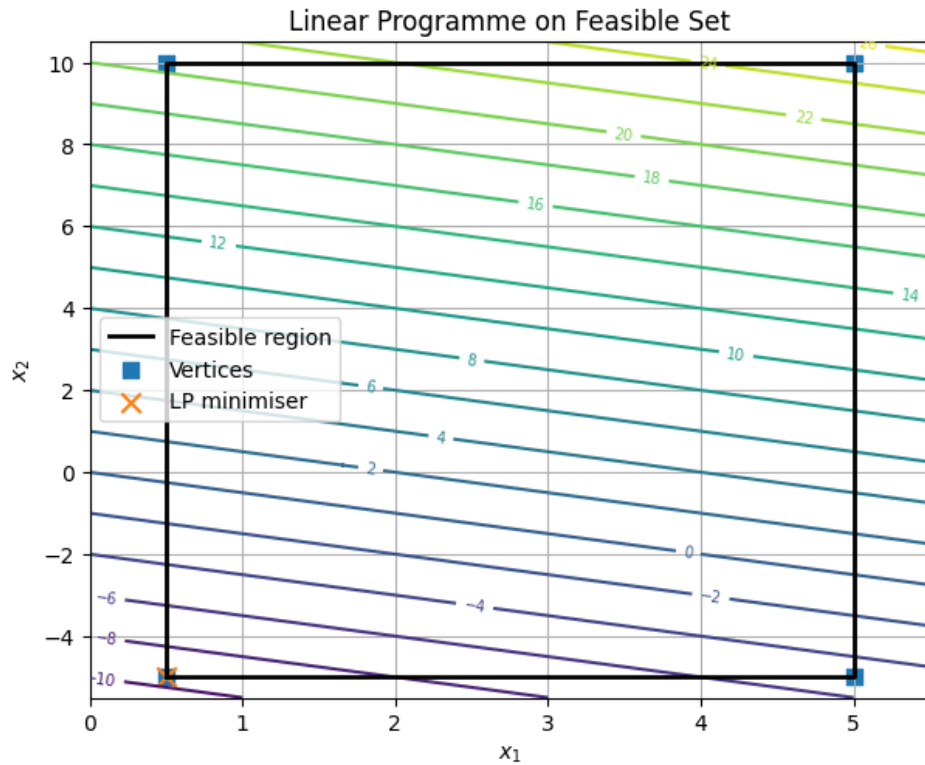
and the linear objective was coded as:

```
a_q6 = np.array([1.0, 2.0])
def f_q6_linear(x):
    return np.dot(a_q6, x)
```

The vertex values were:

$$@ (0.5, -5) = -9.5, \quad @ (5, -5) = -5.0, \quad @ (5, 10) = 25.0, \quad @ (0.5, 10) = 20.5,$$

Therefore, the minimiser is $x^* = (0.5, -5)$, with a minimum value of $f(x^*) = -9.5$.



The contour plot matches the geometry of the problem. Because the coefficient of x_1 is positive, the objective is minimised by taking the smallest feasible x_1 , specifically 0.5. Likewise, because the coefficient of x_2 is also positive, the objective is minimised by taking the smallest feasible x_2 which in this case, is -5. Therefore, the bottom-left vertex is the solution.

(b) Frank–Wolfe: Interior Optimum

The first quadratic objective was

$$f(x) = (x_1 - 1)^2 + (x_2 - 5)^2,$$

whose unconstrained minimiser is (1, 5), which lies in the interior of the box X .

The Frank–Wolfe linear subproblem at each iteration is

$$z_k \in \arg \min_{z \in X} \nabla f(x_k)^\top z.$$

Since X is a box, this linear subproblem can be solved coordinate-wise. This was implemented as the function:

```
def linear_subproblem_box(grad):
    z = np.zeros(2)

    if grad[0] >= 0:
        z[0] = 0.5
    else:
        z[0] = 5.0

    if grad[1] >= 0:
        z[1] = -5.0
    else:
        z[1] = 10.0

    return z
```

The Frank–Wolfe update is derived using the equation:

$$x_{k+1} = \beta x_k + (1 - \beta)z_k,$$

Which was also implemented as a function:

```
def frank_wolfe_box(f, grad_f, x0, beta, n_iters):
    x = x0.copy()

    xs = [x.copy()]
    zs = []
    vals = [f(x)]

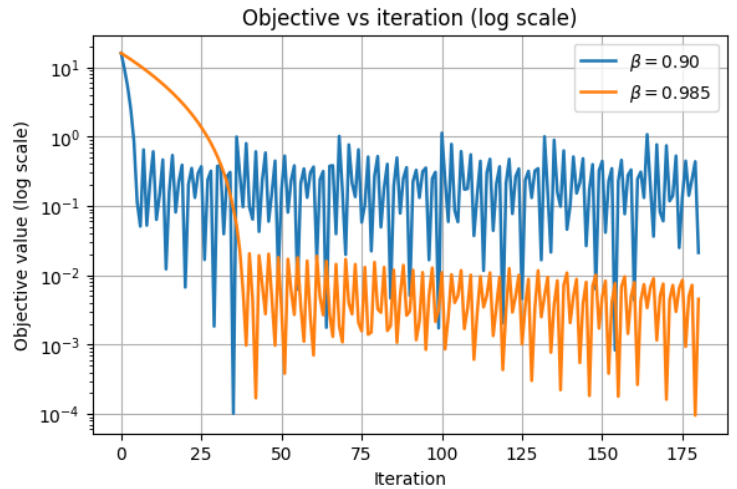
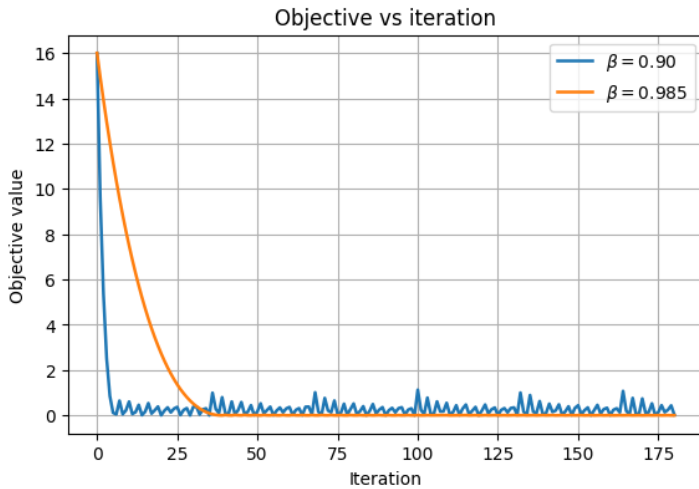
    for _ in range(n_iters):
        g = grad_f(x)
        z = linear_subproblem_box(g)
        x = beta * x + (1.0 - beta) * z

        zs.append(z.copy())
        xs.append(x.copy())
        vals.append(f(x))

    return np.array(xs), np.array(zs), np.array(vals)
```

Two values of β were tested, namely 0.90 and 0.985, starting from $x^{(0)} = (1,1)$.

Interior Optimum Case: Frank-Wolfe Convergence



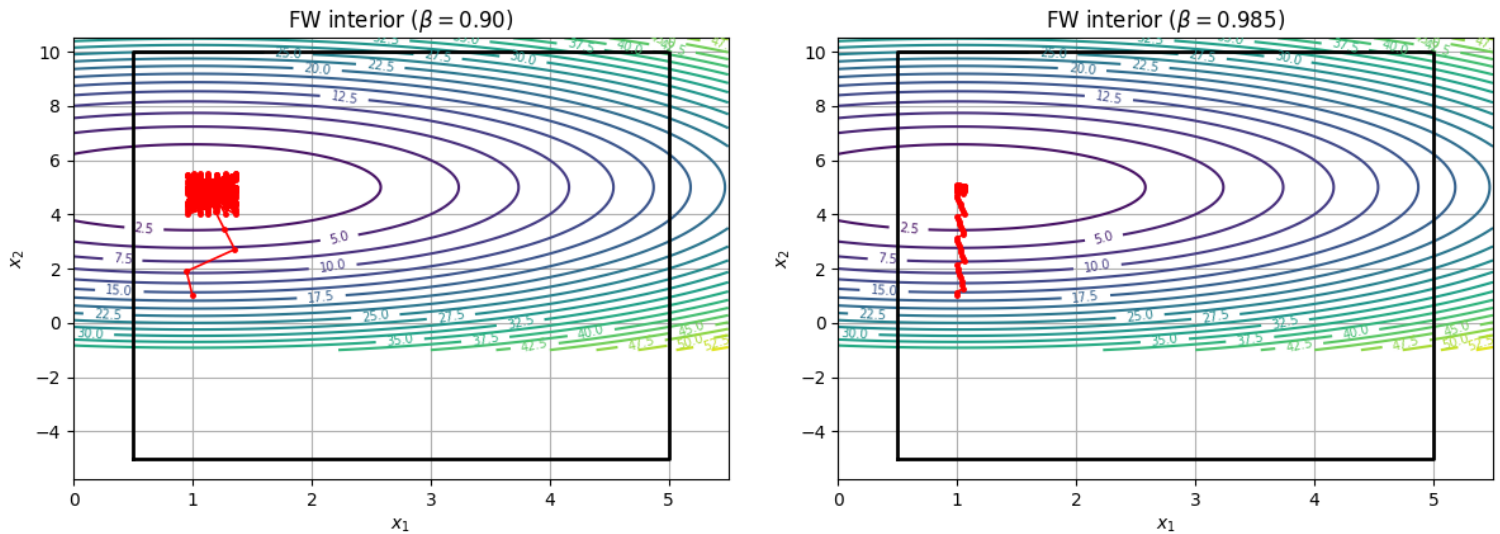
For $\beta = 0.90$, the objective decreased from 16.0 to 0.021 after 180 iterations, with final iterate approximately (1.129, 4.932). The initial objective values were 16.0, 9.613, 5.37, 2.509, 0.856. This shows a relatively fast early decrease. However, the later iterates continued to oscillate around the interior optimum, and the final objective remained at 0.021. The last few values were 0.446, 0.141, 0.245, 0.437, 0.021, showing noticeable oscillation.

For $\beta = 0.985$, the objective decreased from 16.0 to 0.0045, with final iterate approximately (0.9984, 5.0674). The first few objective values were 16.0, 14.938, 13.931, 12.970, 12.056, so the early progress was slower than for $\beta = 0.90$. However, although the final objective was smaller by the end, the method stayed closer to the optimum in the long run. Its final objective was 0.0045, with last few values 0.0009, 0.0050, 0.0072, 0.0001, 0.0045.

Therefore, the convergence curves show a clear trade-off as $\beta = 0.90$ gives faster movement at the start, and $\beta = 0.985$ gives more refined convergence later. This is consistent with the Frank-Wolfe update formula. A larger β gives a smaller step toward the vertex z_k , so the movement is more cautious but less aggressive.

(c) Frank–Wolfe Contour Trajectories

Interior Optimum Case: Frank-Wolfe Trajectories



For the interior-optimum case, the contour plots show that the iterates do not move directly to (1, 5). Instead, the Frank–Wolfe updates repeatedly combine the current point with a vertex of the box, so the trajectory zigzags through the feasible region. This is expected, since the method always moves toward a solution of the linearised subproblem, and over a box that solution lies at a vertex.

For $\beta = 0.90$, the trajectory moves more aggressively between regions of the box, which explains the faster early decrease but also the larger oscillation near the optimum. For $\beta = 0.985$, the trajectory is more conservative and remains closer to the optimum once it gets near it.

The second quadratic objective was

$$f(x) = x_1^2 + x_2^2,$$

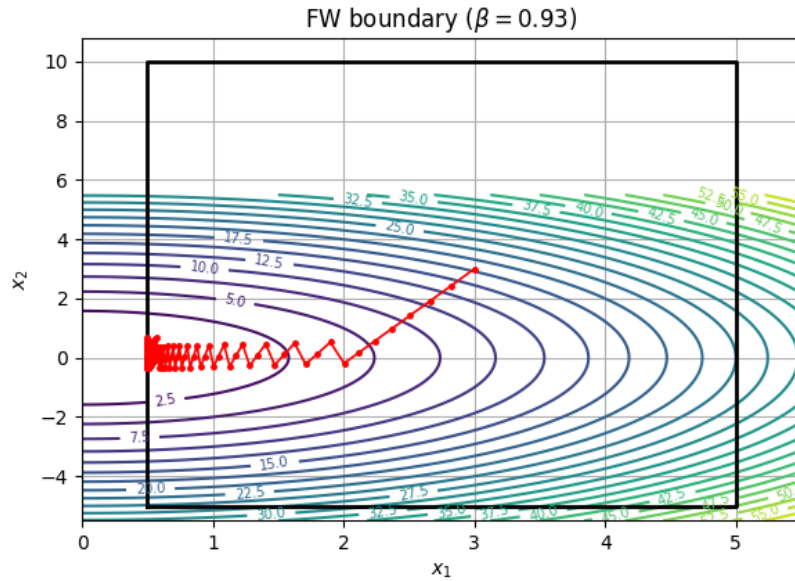
whose unconstrained minimiser is (0,0). Since $x_1 \geq 0.5$ in the feasible set, the constrained optimum lies on the boundary, at

$$x^* = (0.5, 0).$$

For this boundary-optimum case, Frank–Wolfe was run with $\beta = 0.93$, starting from $x^{(0)} = (3, 3)$. The objective decreased from 18.0 to 0.364, with final iterate approximately (0.5001, -0.337). The minimum objective value reached during the run was 0.250, which is close to the true constrained optimum value:

$$f(0.5, 0) = 0.25.$$

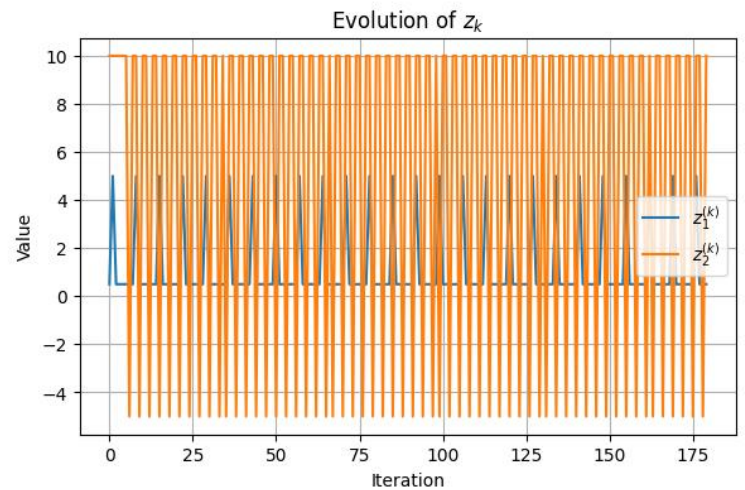
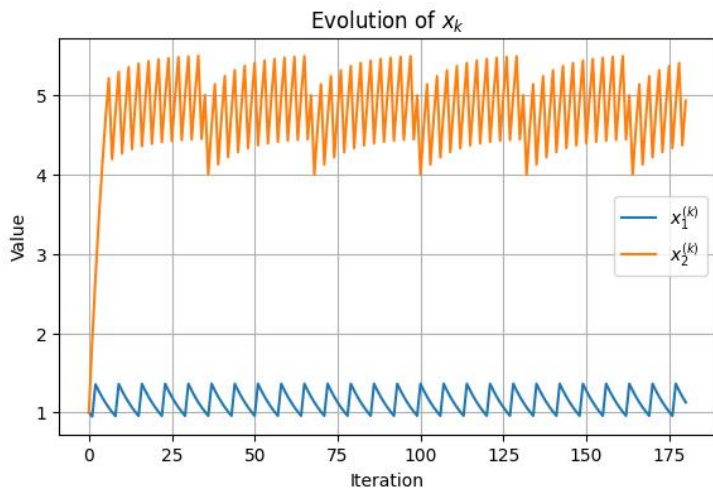
Boundary Optimum Case: Frank-Wolfe Trajectory



The contour plot shows that the iterates approach the boundary $x_1 = 0.5$ and then oscillate around the boundary-optimal region rather than settling exactly at $(0.5, 0)$. This is standard for Frank-Wolfe when using a fixed convex-combination parameter β , since the iterates are repeatedly pulled toward vertices of the feasible set rather than projected directly onto the optimum.

(d) Evolution of x_k and z_k vs Iteration

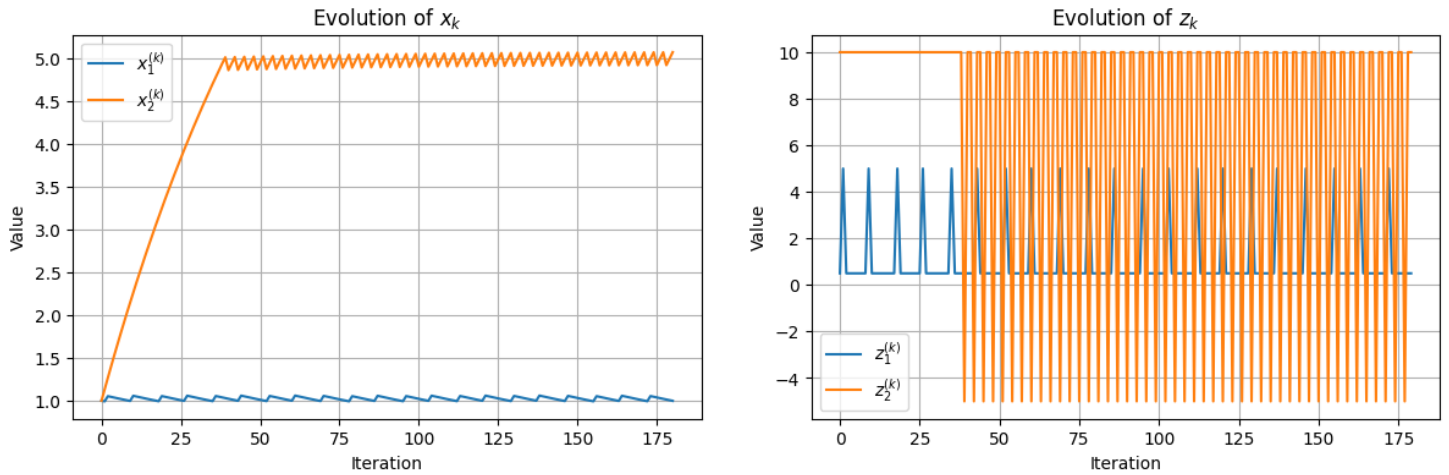
Interior Optimum Case: x_k and z_k Evolution ($\beta = 0.90$)



In the interior-optimum case with $\beta = 0.90$, the iterates x_k moved substantially in both coordinates. The final point was $(1.129, 4.932)$, but the last few values still oscillated. For example, the last few x_1 values were approximately 0.958, 1.363, 1.276, 1.199, 1.129, and the last few x_2 values were 4.334, 4.90, 5.410, 4.369, 4.932. This shows that the method keeps switching between different linear subproblem solutions z_k , producing a zig-zag pattern.

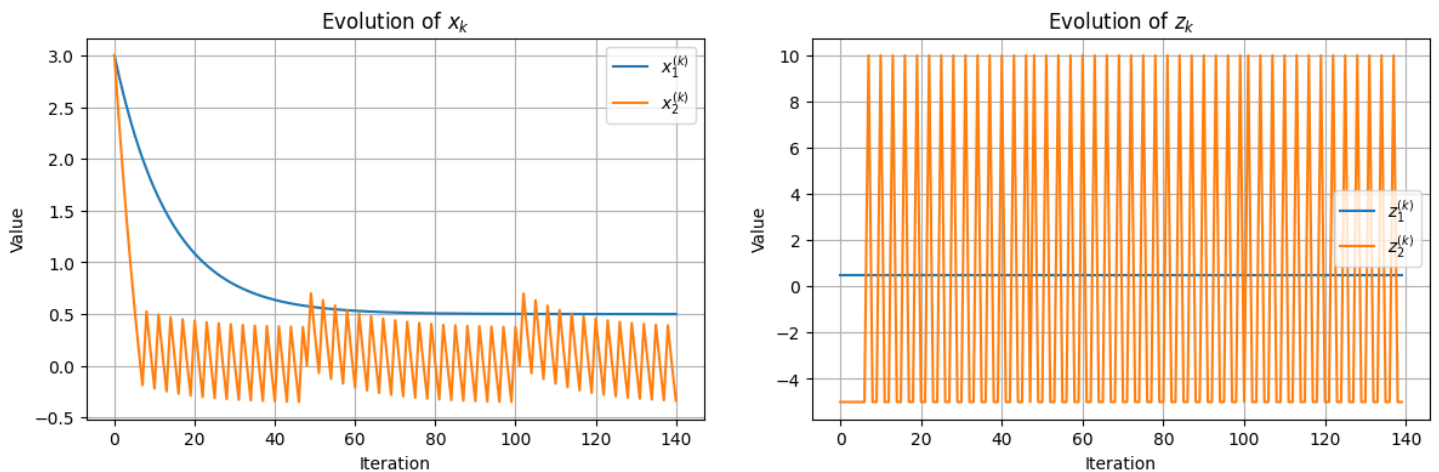
For $\beta = 0.985$, the same effect occurs, but the oscillations are smaller. The last few x_1 values were 1.03, 1.022, 1.014, 1.006, 0.998, and the last few x_2 values were 4.992, 5.067, 4.916, 4.992, 5.067. So, the iterates remain closer to the optimum.

Interior Optimum Case: x_k and z_k Evolution ($\beta = 0.985$)



In the boundary-optimum case, the x_k evolution shows that x_1 converges close to the boundary value 0.5, while x_2 keeps oscillating around zero. The x_1 values fluctuate around 0.5001 in its latter iterations, so the boundary constraint is effectively active. The x_2 values, however, were 0.0194, -0.332, 0.391, 0.014, -0.337, which shows that the iterates continue to move above and below zero.

Boundary Optimum Case: x_k and z_k Evolution ($\beta = 0.93$)



The z_k plots explain this. Since the linear subproblem is solved over a box, each z_k is always a vertex of the feasible region. Therefore, the algorithm repeatedly chooses vertices that minimise the current linearisation, and the iterate x_k is just a convex combination of past points and these chosen vertices. This is why the method does not settle smoothly but instead keeps moving in a vertex-driven way.

(e) Interior vs Boundary Optimum Behaviour

The interior-optimum and boundary-optimum cases illustrate two different behaviours of Frank-Wolfe. In the interior-optimum case, the true minimiser (1,5) lies strictly inside the feasible set, but the algorithm still only uses vertices of the feasible set in its linear subproblem. As a result, the iterates approach the optimum indirectly through convex combinations of boundary points. This causes oscillation, especially when the step toward z_k is relatively large, as in the $\beta = 0.90$ case.

In the boundary-optimum case, the true constrained solution itself lies on the boundary of the feasible set. Here the method is better aligned with the geometry of the problem, since the solution is already near one of the extreme directions of the box. Even so, because the update still depends on box vertices, the iterates oscillate around the optimum rather than settling exactly at it.

This investigation demonstrates that Frank-Wolfe is well suited to constrained optimisation over simple feasible sets such as boxes, since the linear subproblem is easy to solve. However, the method behaves differently depending on whether the optimum lies in the interior or on the boundary. In the interior case, the method may zig-zag more noticeably because the optimum is not itself an extreme point. In the boundary case, the behaviour is usually more aligned with the feasible geometry, although oscillation can remain when a fixed β is used.